SE507 - Software Testing and Evaluation

A Comprehensive Testing Strategy for a Library Management System

Team Members:

Tishko Salah `(Test Manager)`
Sara Zuhair `(Code specifier)`
Yassin Hussein `(Code reviewer)`
Ravyar Barzan `(Document inspector)`

Module Leader: **Dr. Hossein Hassani**

June 15, 2025

University of Kurdistan Hewlêr
Department of Computer Science and Engineering
Erbil, Kurdistan Region, Iraq

Contents

List of Figures

List of Tables

Background

This chapter provides a comprehensive overview of the Library Management System, the System Under Test (SUT). It details the system's purpose, architecture, user roles, core functionalities, and underlying data model. This context is essential for understanding the scope, strategy, and execution of the testing activities detailed in subsequent chapters.

1.1 System Under Test (SUT) Overview

The Library Management System is a full-stack web application designed to modernize and streamline library operations **srs**. Its primary purpose is to provide a digital platform for managing library resources, including books, authors, and genres, while also managing user accounts for both librarians and library members **srs**. The system aims to replace or supplement traditional, manual library management methods with an efficient, accessible, and user-friendly web-based solution **srs**.

The project's scope encompasses a complete, self-contained system featuring a backend API, a database, and a frontend client application **srs**. Key capabilities include:

- **User Authentication:** Secure user registration, login, and logout functionalities with role-based access control for two distinct user types: Librarians and Members **srs**.
- **Inventory Management:** Comprehensive CRUD (Create, Read, Update, Delete) operations for the core library entities: books, authors, and genres **srs**.
- **Borrowal Tracking:** Management of book borrowals and returns, allowing librarians to oversee lending activities and members to view their history **srs**.
- **User Administration:** Tools for librarians to manage all user accounts within the system **srs**.
- **Review System:** A newly introduced feature allowing members to submit reviews for books, which can then be managed by librarians **srs**.

1.2 System Architecture

The system is built on a modern, multi-tier architecture, which separates concerns between the client, server, and database. This design promotes modularity, scalability, and maintainability.

- **Frontend (Client-Side):** The user interface is a single-page application (SPA) developed using **React** and the **Material-UI (MUI)** component library. It is responsible for rendering the UI and interacting with the backend via API calls. It runs in any modern web browser **srs**.

- **Backend (Server-Side):** The backend is a RESTful API built with **Node.js** and the **Express.js** framework. It handles all business logic, data processing, and user authentication using Passport.js **srs**.
- **Database:** Data persistence is managed by a **MongoDB** database, a NoSQL database that stores data in flexible, JSON-like documents. The backend interacts with MongoDB via the Mongoose ODM library **srs**.

The entire application is containerized using **Docker**, ensuring consistent development, testing, and deployment environments **srs**. Figure 1.1 illustrates the high-level architecture.

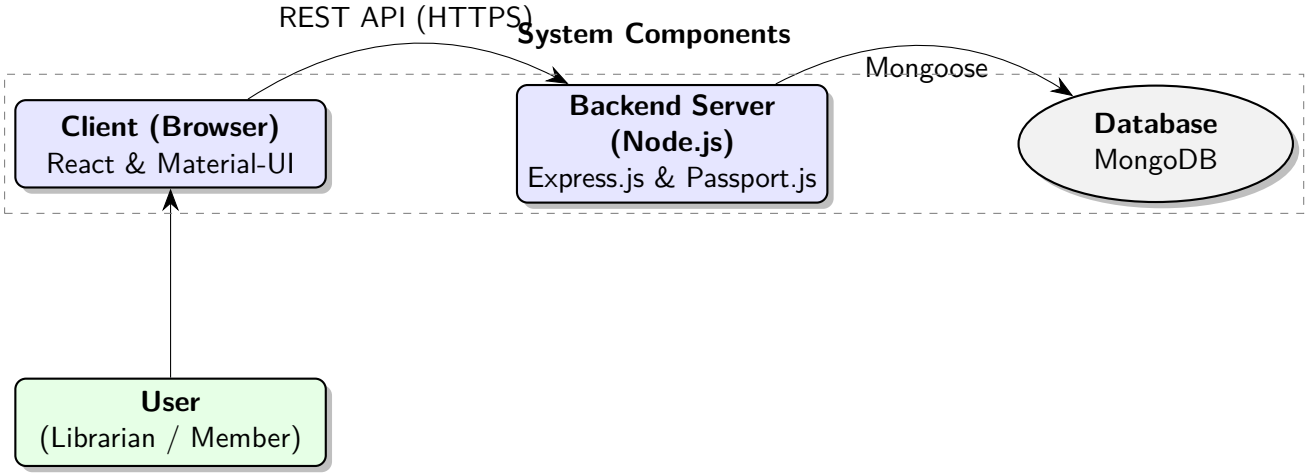

Figure 1.1: High-Level System Architecture.

1.3 User Roles and Functionalities

The system defines two distinct user roles, each with a specific set of permissions and responsibilities as outlined in the SRS **srs**.

1.3.1 Librarian (Admin)

Librarians are the administrators of the system with full control over all its aspects. Their primary responsibilities include:

- Managing the library's entire inventory, including adding, updating, and deleting books, authors, and genres **srs**.
- Overseeing all user accounts, which involves creating new accounts for both members and other librarians, as well as updating and deleting existing accounts **srs**.
- Managing the complete lifecycle of book borrowals, from creation to marking returns **srs**.
- Viewing system-wide statistics and data on a dedicated dashboard **srs**.
- Moderating book reviews submitted by members **srs**.

1.3.2 Member

Members are the standard users of the library. Their access is generally limited to viewing data and managing their own activities. Their functions include:

- Browse the library's catalog of books, authors, and genres **srs**.

- Requesting to borrow available books **srs**.
- Viewing their personal borrowal history **srs**.
- Submitting, viewing, and managing their own reviews for books **srs**.

1.4 Data Model

The system's data is structured around several core entities, whose relationships define the application's functionality. The main entities are **User**, **Book**, **Author**, **Genre**, **Borrowal**, and **Review srs**. The relationships between these entities are a key focus of integration testing to ensure data integrity. For instance, a *Borrowal* record links a *User* to a *Book*, and a *Book* can have one *Author* and one *Genre* **srs**.

Figure 1.2 provides a conceptual Entity-Relationship Diagram (ERD) illustrating these connections.

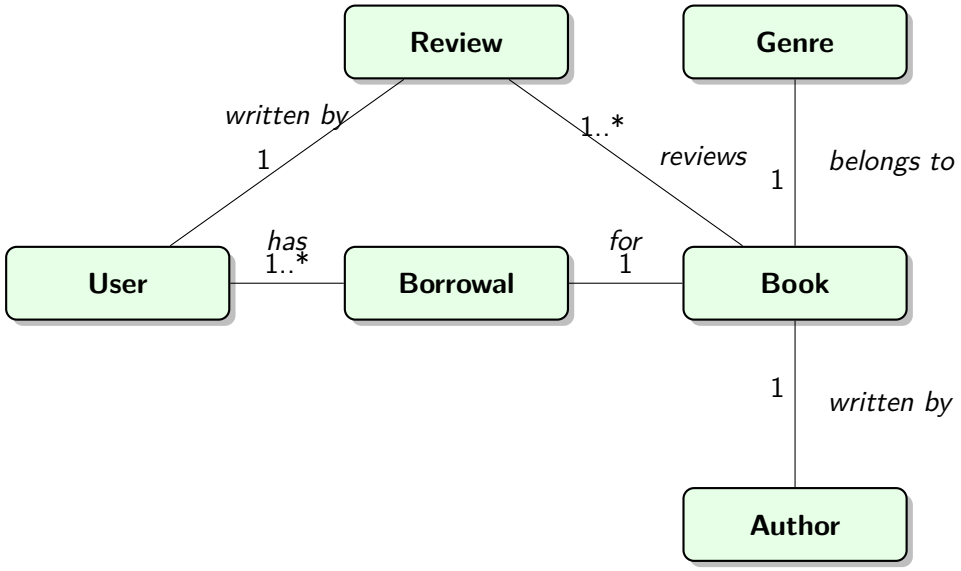

Figure 1.2: Conceptual Entity-Relationship Diagram (ERD).

Test Management

This section outlines the comprehensive management plan for the testing of the Library Management System. The plan is designed to ensure that all software components are rigorously tested, meet the requirements specified in the Software Requirements Specification (SRS), and achieve the highest standards of quality. This document details the test strategy, scope, team roles, environment, schedule, deliverables, risk management, and success criteria for the project.

2.1 Test Strategy

Our test strategy employs a multi-layered approach, integrating various testing levels to ensure comprehensive coverage and early defect detection. The strategy is aligned with the V-Model, ensuring that testing activities are planned in parallel with development phases.

2.1.1 Overall Approach

The testing process is structured across multiple levels:

- **Static Testing:** Performed without executing the code, focusing on reviews and analysis of documentation and source code to find defects early.
- **Unit Testing:** Developers will test individual components (functions, classes) in isolation to verify their correctness.
- **Integration Testing:** Focuses on testing the interfaces and interactions between integrated components, such as the communication between the frontend client and the backend API.
- **System & E2E Testing:** The complete, integrated system is tested from end-to-end to validate that it meets all specified requirements from a user's perspective.
- **Non-Functional Testing:** Ensures the system meets requirements for performance, security, usability, and compatibility.
- **Regression Testing:** Conducted after modifications to ensure that new changes have not introduced new defects into existing functionality.

2.1.2 Test Levels

- **Unit Testing:** Implemented using **Jest**. For the backend, this involves testing Node.js controllers, models, and utility functions. For the frontend, this includes testing React components, hooks, and utility functions using Jest and the React Testing Library.

- **API & Integration Testing:** Backend API endpoints are tested using **Jest** and **Supertest** to verify CRUD operations, authentication flows, and business logic. This also covers the integration points between different services.
- **End-to-End (E2E) Testing:** Real user workflows are simulated using **Cypress**. These tests cover complete user journeys, such as logging in, searching for a book, borrowing it, and logging out.
- **Non-Functional Testing:**
  - **Performance Testing: K6** is used to script and execute load tests against the API to measure response times, throughput, and stability under load.
  - **Security Testing:** Manual and automated checks are performed to identify common vulnerabilities (e.g., SQL injection, XSS, insecure direct object references). Cypress tests are used to check for authorization flaws.
  - **Usability & Compatibility Testing:** Manual testing and Cypress scripts ensure the application is user-friendly and renders correctly across major web browsers (Chrome, Firefox, Safari).

2.2 Test Scope

2.2.1 In Scope

The following features and functionalities, as defined in the SRS, are within the scope of testing:

- User Authentication (Login, Logout, Role-based Access Control)
- Librarian Dashboard and statistical widgets
- Full CRUD (Create, Read, Update, Delete) operations for Books, Authors, and Genres
- Borrowal Management (borrowing, returning, and tracking books)
- User Management (Librarian managing member accounts)
- Book Review and Rating system
- API endpoint functionality and security
- Database referential integrity

2.2.2 Out of Scope

The following are considered out of scope for testing:

- Testing of third-party libraries and frameworks (e.g., React, Express, Mongoose).
- Performance of the underlying hardware, network infrastructure, or MongoDB database engine itself.
- Exhaustive testing of every possible combination of user inputs.
- Usability testing on mobile operating systems (unless specified).

2.3 Team Roles and Responsibilities

The testing process involves the entire team, with specific roles and responsibilities as defined in the project's task distribution plan.

Table 2.1: Team Roles and Primary Responsibilities

Member	**Role**	**Primary Testing Responsibilities**
Tishko Salah	Test Manager	Oversees the entire testing process, manages the test plan and schedule, coordinates team efforts, and is the final authority on test completion.
Sara Zuhair	Code Specifier	Focuses on non-coder testing activities, including functional validation against SRS, documentation review, and usability testing. Leads the execution of user-centric test cases.
Yassin Hussein	Code Reviewer	Leads technical testing activities, including unit testing, API testing, and code reviews. Ensures code quality, performance, and adherence to standards.
Ravyar Barzan	Document Inspector	Leads the review and inspection of all project documentation (SRS, Test Plan, Test Cases, Final Report) for clarity, consistency, and completeness.

2.4 Test Environment and Tools

All testing is conducted within containerized environments managed by Docker to ensure consistency and reproducibility.

- **Operating Systems:** Windows, macOS, and Linux (for development and execution).
- **Test Environments:** Defined in `docker-compose.dev.yml`, `docker-compose.test.yml`, and `docker-compose.prod.yml` to simulate development, testing, and production setups.
- **Version Control:** Git, with GitHub for repository hosting and collaboration.

Table 2.2: Testing Tools and Frameworks

Category	**Tool/Framework**	**Purpose**
Backend Testing	Node.js, Jest, Supertest	Unit and integration testing of API endpoints and services.
Frontend Testing	Jest, React Testing Library	Unit and component testing of React components and logic.
E2E Testing	Cypress	Automated end-to-end testing of user workflows in a browser.
Performance	K6, Docker	Load testing the backend API to measure performance metrics.
Code Quality	ESLint, Prettier	Static code analysis to enforce coding standards and find bugs.
Documentation	LaTeX, Draw.io	Creating project reports, diagrams, and test artifacts.

2.5 Test Schedule

The project timeline spans approximately 14 weeks, from early March to mid-June 2025. The schedule is visualized in the Gantt chart below (Figure 2.1).

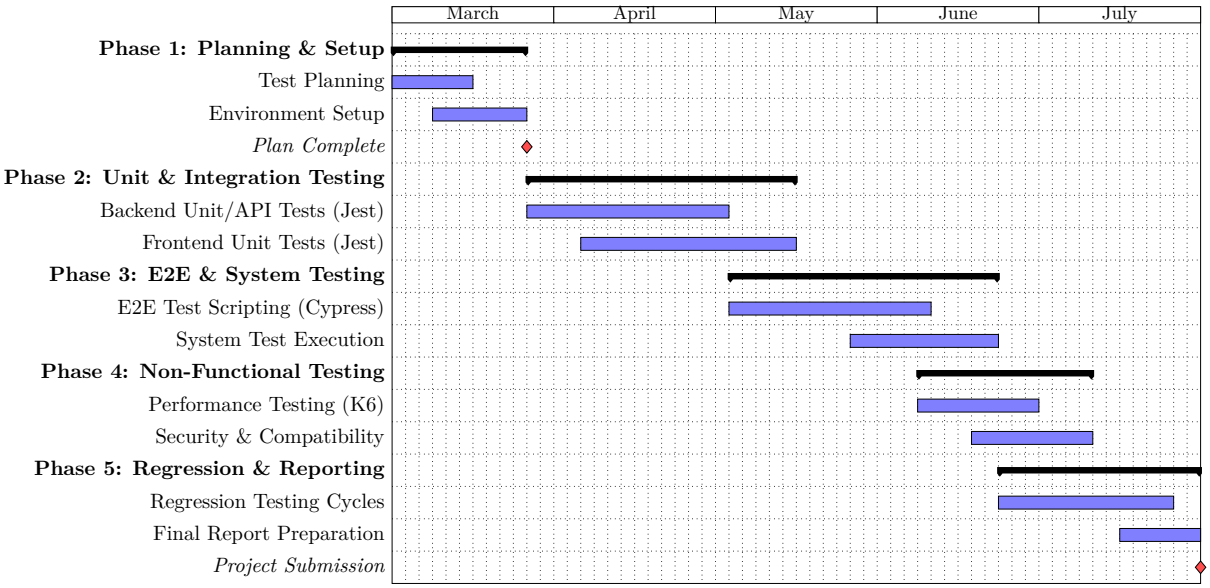

Figure 2.1: Project Testing Schedule Gantt Chart

2.6 Deliverables

The following artifacts will be produced as part of the testing process:

- **Test Plan:** This comprehensive document (of which this chapter is a part).
- **Test Case Documentation:** Detailed test cases for all levels of testing, located in `/docs/Report/test_artifacts/test_cases/`.
- **Test Scripts:** All automated test scripts written using Jest, Cypress, and K6.
- **Defect Reports:** Bugs and issues will be tracked, prioritized, and managed. (A system like GitHub Issues will be used).
- **Test Execution Reports:** Automated reports generated by Jest (`/server/test-report/report.html`) and Cypress (videos/screenshots) after test runs.
- **Final Test Summary Report:** The complete final report detailing all testing activities, results, and conclusions.

2.7 Risk Management

Potential risks that could impact the testing schedule and quality are identified and managed proactively.

Table 2.3: Risk Assessment and Mitigation Plan

Risk Description	Likelihood	Impact	Mitigation Strategy
Delays in development phases impact testing start dates.	Medium	High	Maintain close communication between developers and testers. Adapt the test plan and prioritize testing of critical features.
Test environment is unstable or differs from production.	Low	High	Use Docker to ensure consistent environments. Perform smoke tests on deployment to validate environment stability.
Critical defects are discovered late in the project timeline.	Medium	High	Emphasize early testing (unit, integration). Prioritize test cases based on risk and critical user workflows.
Team member unavailability due to unforeseen circumstances.	Low	Medium	Ensure knowledge is shared across the team. Cross-train members on critical testing tasks and tools.
Underestimation of time required for test case creation or execution.	Medium	Medium	Base estimates on feature complexity. Regularly review progress and re-allocate resources as needed.

2.8 Entry and Exit Criteria

Clear criteria define the start and end points for the formal testing process.

2.8.1 Entry Criteria

Testing on a feature or build can begin when:

- The SRS and relevant design documents are complete and approved.
- The code has been developed, peer-reviewed, and successfully built.
- All developer-written unit tests for the feature are passing.
- The required test environment and test data are available and stable.
- Test cases have been written, reviewed, and approved by the test manager.

2.8.2 Exit Criteria

The overall testing phase will be considered complete when:

- All planned test cases have been executed.
- A pass rate of 100% for all critical and major test cases is achieved.
- A pass rate of over 95% for minor test cases is achieved.
- No outstanding Severity-1 (blocker) or Severity-2 (critical) defects remain open.

- All major functional and non-functional requirements are verified and met.
- The final test report is completed and approved by all team members.

Testing and Evaluation Process

This section details the comprehensive testing and evaluation process designed to ensure the Library Management System (LMS) meets the specified requirements and quality standards. Our strategy employs a multi-layered approach, combining static and dynamic testing techniques across various levels of the application architecture. The process is structured to systematically identify, document, and resolve defects throughout the software development lifecycle (SDLC).

The primary objectives of our testing process are to:

- Verify that all functional requirements outlined in the Software Requirements Specification (SRS) are correctly implemented.
- Ensure that the system meets all non-functional requirements, including performance, security, and usability benchmarks.
- Validate the system's stability, reliability, and robustness through rigorous integration, regression, and end-to-end testing.
- Provide comprehensive test coverage reports and defect analyses to stakeholders for informed decision-making.

3.1 Testing Strategy Overview

Our testing strategy is built upon a foundation of continuous testing, integrating quality assurance activities into every phase of the development cycle. The strategy is visualized in Figure 3.1. It begins with static testing techniques like code and documentation reviews, followed by a hierarchy of dynamic testing levels, from unit testing of individual components to system-wide end-to-end testing.

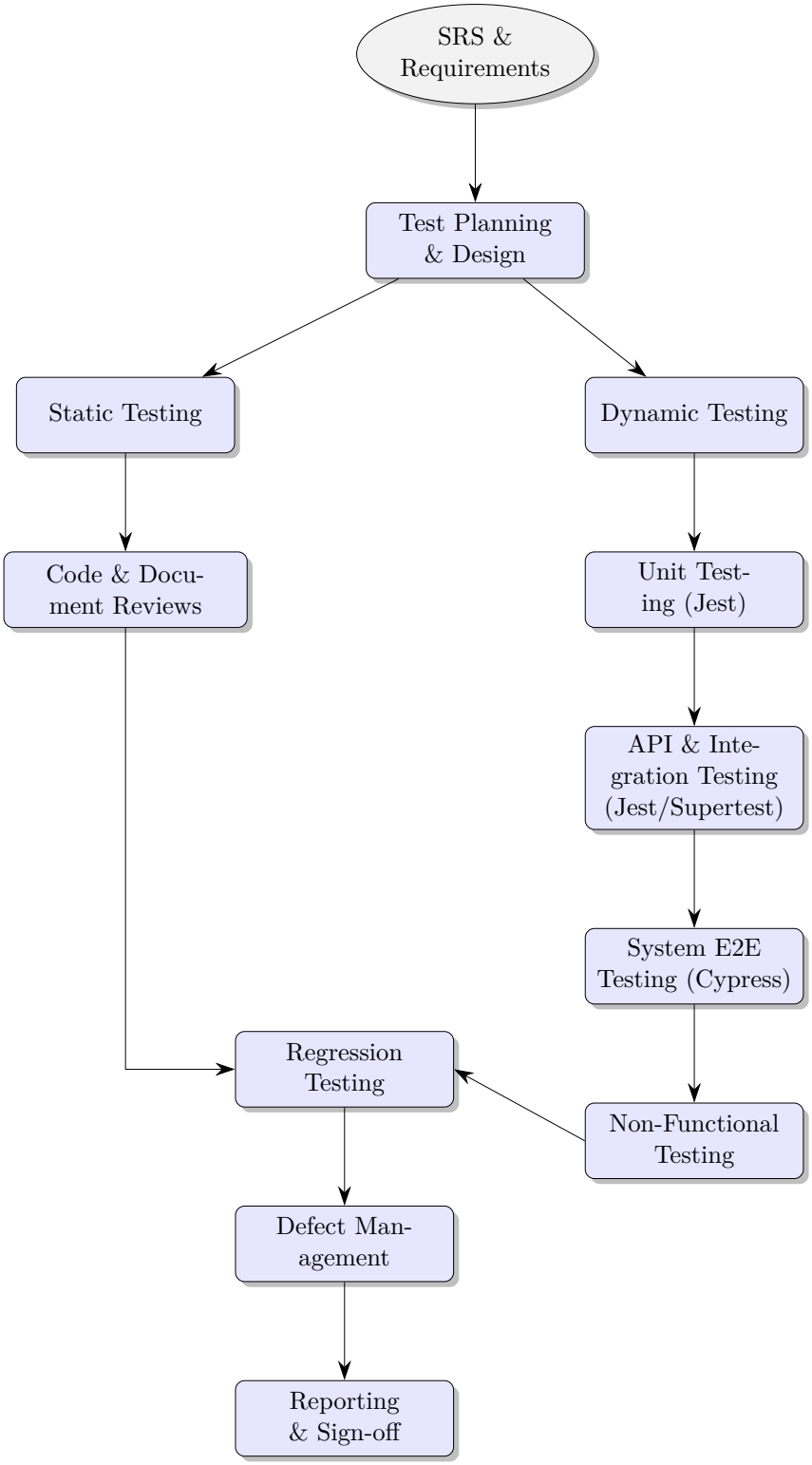

Figure 3.1: Overall Testing Process Flowchart

3.2 Testing Levels

Testing is structured into distinct levels to ensure all parts of the system are validated, from the smallest code unit to the fully integrated application.

3.2.1 Unit Testing

Unit tests focus on the smallest individual components of the application in isolation. This is primarily aimed at the controllers, models, and utility functions within the Node.js backend.

- **Objective:** To verify that each function or module behaves as expected.
- **Framework:** Jest is used as the testing framework, assertion library, and test runner.
- **Methodology:** Dependencies such as database models or external services are mocked to ensure tests are fast, reliable, and isolated. Test cases cover both successful paths and error conditions.

Detailed unit test cases are documented in the corresponding test file.

3.2.1.1 Authentication Controller (authController)

The Auth Controller is responsible for user authentication, including registration (by a librarian), login, and logout. These tests ensure that access control and user session management are functioning correctly, as specified in the SRS.

Table 3.1: Test Cases for authController

Test Case ID	**Related Requirement (SRS)**	**Test Scenario**	**Test Steps**	**Expected Result**
TC-AUTH-001	3.1.1 User Registration , UC-004	Successful user registration	1. Mock `User.findOne` to resolve as null. 2. Mock `User.create` to resolve with a new user object. 3. Call `registerUser` with valid new user data.	1. Response status is 201. 2. Response JSON indicates successful registration.
TC-AUTH-002	UC-004 (A1: Email Already Exists)	Attempt to register a user that already exists	1. Mock `User.findOne` to resolve with an existing user object. 2. Call `registerUser` with data of an existing user.	1. Response status is 400. 2. Response JSON indicates that the username or email already exists.

Continued on next page

Table 3.1 – continued from previous page

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-AUTH-003	3.1.2 User Login , UC-001	Successful user login with valid credentials	1. Mock `passport.authenticate` to succeed and return a user object. 2. Call `loginUser` middleware with correct username and password.	1. `req.logIn` is called. 2. Response status is 200. 3. Response JSON indicates successful login and includes user data.
TC-AUTH-004	UC-001 (A1: Invalid Credentials)	Attempt to login with invalid credentials	1. Mock `passport.authenticate` to fail (returns false). 2. Call `loginUser` middleware with an incorrect username or password.	1. Response status is 401. 2. Response JSON indicates incorrect credentials.
TC-AUTH-005	3.1.3 User Logout	Successful user logout	1. Mock `req.logout`. 2. Call `logoutUser`.	1. `req.logout` is called. 2. Response status is 200. 3. Response JSON indicates successful logout.

3.2.1.2 Author Controller (authorController)

The Author Controller manages CRUD operations for author entities. These tests validate the creation and retrieval of authors.

Table 3.2: Test Cases for authorController

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-ATHR-001	3.4 Author Management (POST /api/author/add)	Successfully create a new author	1. Provide author details in `req.body`. 2. Mock `Author.create` to resolve with the new author data. 3. Call `createAuthor`.	1. Response status is 201. 2. Response JSON contains the newly created author.
TC-ATHR-002	3.4 Author Management (GET /api/author/getAll)	Successfully retrieve all authors	1. Mock `Author.find` to resolve with an array of author objects. 2. Call `getAllAuthors`.	1. Response status is 200. 2. Response JSON contains the array of all authors.

3.2.1.3 Book Controller (bookController)

The Book Controller handles all CRUD operations for book entities. The tests ensure that books can be created, retrieved (all and by ID), updated, and deleted as per the requirements.

Table 3.3: Test Cases for bookController

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-BOOK-001	3.3 Book Management (POST /api/book/add) , UC-002	Successfully create a new book	1. Provide book details in `req.body`. 2. Mock `Book.create` to resolve successfully. 3. Call `createBook`.	1. Response status is 201. 2. Response JSON contains the new book.
TC-BOOK-002	UC-002 (A2: API Error)	Fail to create a book due to a database error	1. Mock `Book.create` to reject with an error. 2. Call `createBook`.	1. Response status is 500. 2. Response JSON contains an error message.
TC-BOOK-003	3.3 Book Management (GET /api/book/getAll)	Successfully retrieve all books	1. Mock `Book.find().populate()` to resolve with an array of books. 2. Call `getAllBooks`.	1. Response status is 200. 2. Response JSON contains the array of books.
TC-BOOK-004	3.3 Book Management (GET /api/book/get/:id)	Successfully retrieve a single book by its ID	1. Provide a book ID in `req.params`. 2. Mock `Book.findById().populate()` to resolve with a book object. 3. Call `getBookById`.	1. Response status is 200. 2. Response JSON contains the specific book.
TC-BOOK-005	3.3 Book Management (GET /api/book/get/:id)	Attempt to retrieve a book with a non-existent ID	1. Provide a non-existent book ID in `req.params`. 2. Mock `Book.findById().populate()` to resolve as null. 3. Call `getBookById`.	1. Response status is 404. 2. Response JSON contains a 'Book not found' message.
TC-BOOK-006	3.3 Book Management (PUT /api/book/update/:id)	Successfully update an existing book	1. Provide a book ID and update data. 2. Mock `Book.findByIdAndUpdate` to resolve with the updated book. 3. Call `updateBook`.	1. Response status is 200. 2. Response JSON contains the updated book data.

Continued on next page

Table 3.3 – continued from previous page

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-BOOK-007	3.3 Book Management (DELETE /api/-book/delete/:id)	Successfully delete a book	1. Provide a book ID to be deleted. 2. Mock `Book.findByIdAndDelete` to resolve successfully. 3. Call `deleteBook`.	1. Response status is 200. 2. Response JSON contains a 'Book removed' message.

3.2.1.4 Borrowal Controller (borrowalController)

This controller manages borrowal records. Tests cover creating a borrowal (and updating book availability), updating a borrowal's status, and retrieving borrowals based on user roles (Librarian vs. Member).

Table 3.4: Test Cases for borrowalController

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-BORR-001	3.6 Borrowal Management , UC-003	Create a borrowal for an available book	1. Mock `Book.findById` to return an available book. 2. Mock `Borrowal.create` to succeed. 3. Call `createBorrowal`.	1. Book's availability is set to false. 2. Response status is 201. 3. Response JSON contains the new borrowal record.
TC-BORR-002	UC-003 (A1: Book Not Available)	Attempt to create a borrowal for an unavailable book	1. Mock `Book.findById` to return an unavailable book. 2. Call `createBorrowal`.	1. Response status is 400. 2. Response JSON has a "Book is not available" message.
TC-BORR-003	3.6 Borrowal Management (PUT /api/borrowal/update/:id)	Update borrowal status to 'Returned'	1. Mock `Borrowal.findById` to find a borrowal. 2. Mock `Book.findByIdAndUpdate` to succeed. 3. Call `updateBorrowal` with status 'Returned'.	1. Book's availability is set to true. 2. Response status is 200. 3. Response JSON contains the updated borrowal.

Continued on next page

Table 3.4 – continued from previous page

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-BORR-004	3.6 Borrowal Management (GET /api/borrowal/getAll)	Get all borrowals as a Librarian	1. Set `req.user.role` to 'Librarian'. 2. Mock `Borrowal.find` to return all borrowals. 3. Call `getBorrowals`.	1. `Borrowal.find` is called with an empty filter. 2. Response contains all borrowal records.
TC-BORR-005	3.6 Borrowal Management , UC-005	Get own borrowals as a Member	1. Set `req.user.role` to 'Member'. 2. Mock `Borrowal.find` to return member-specific borrowals. 3. Call `getBorrowals`.	1. `Borrowal.find` is called with a filter for the member's ID. 2. Response contains only the member's borrowals.

3.2.1.5 Genre Controller (genreController)

The Genre Controller manages CRUD operations for genre entities. These tests validate the creation and retrieval of genres.

Table 3.5: Test Cases for genreController

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-GENR-001	3.5 Genre Management (POST /api/genre/add)	Successfully create a new genre	1. Provide genre details in `req.body`. 2. Mock `Genre.create` to resolve with the new genre data. 3. Call `createGenre`.	1. Response status is 201. 2. Response JSON contains the newly created genre.
TC-GENR-002	3.5 Genre Management (GET /api/genre/getAll)	Successfully retrieve all genres	1. Mock `Genre.find` to resolve with an array of genre objects. 2. Call `getAllGenres`.	1. Response status is 200. 2. Response JSON contains the array of all genres.

3.2.1.6 Review Controller (reviewController)

The Review Controller is responsible for managing book reviews. The tests cover the creation of reviews and fetching all reviews for a specific book.

Table 3.6: Test Cases for reviewController

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-REV-001	3.8 Review Management (POST /api/review/add)	Successfully create a review for a book	1. Provide review details (bookId, rating, comment) in `req.body`. 2. Mock `Review.create` to resolve with the new review object. 3. Call `createReview`.	1. Response status is 201. 2. Response JSON contains the newly created review.
TC-REV-002	3.8 Review Management	Get all reviews for a specific book	1. Provide a book ID in `req.params.bookId`. 2. Mock `Review.find().populate()` to resolve with an array of reviews. 3. Call `getReviewsForBook`.	1. `Review.find` is called with a filter for the book ID. 2. Response status is 200. 3. Response JSON contains the array of reviews for that book.

3.2.2 Integration Testing

Integration testing focuses on verifying the interaction between different components. For the LMS, this primarily involves testing the API endpoints to ensure they correctly interact with controllers, models, and the database.

- **Objective:** To detect defects in the interfaces and interactions between integrated components.
- **Framework:** Jest and Supertest are used to send HTTP requests to the API endpoints and assert the responses.
- **Methodology:** Tests cover creating, reading, updating, and deleting (CRUD) entities, as well as complex workflows involving multiple API calls. A test database is used to ensure a clean state for each test run.

The specific test cases for integration testing are detailed in:

Table 3.7: Integration Test Cases

Test Case ID	Test Title / Scenario	Test Steps	Expected Results
Scenario: Full Lifecycle - From Entity Creation to Borrowal			
			Continued on nex

Table 3.7 – continued from previous page

Test Case ID	Test Title / Scenario	Test Steps	Expected Results
TC_INT_001	End-to-end workflow of creating all required entities for a new book, registering a new member, and the member borrowing that book.	• **As Librarian:** Login to the system. • Navigate to Author Management and add a new author (e.g., "J.R.R. Tolkien"). • Navigate to Genre Management and add a new genre (e.g., "Fantasy"). • Navigate to Book Management and add a new book ("The Hobbit"), linking the newly created author and genre. Set ‘isAvailable‘ to true. • Navigate to User Management and register a new user with ‘isAdmin=false‘ (e.g., "Member1"). • Logout. • **As Member1:** Login to the system. • Navigate to the book list and find "The Hobbit". • Initiate and confirm the borrowing of "The Hobbit".	• The author is created successfully and selecta the book form. • The genre is created successfully and selecta the book form. • The book is created wit correct author and genr associations. The book shows its status as avai • The new member accou created and appears in user list. • Member1 can log in successfully and is direc the member landing pa • A new borrowal record created for Member1 ar "The Hobbit". • The ‘isAvailable‘ status "The Hobbit" in the Bo entity is updated to ‘fal • The new borrowal recor appears in Member1’s borrowal history.
Scenario: Data Integrity and Referential Constraints			
TC_INT_002	Verify data integrity when a linked entity (Author) is deleted after being associated with another entity (Book).	• **As Librarian:** Login. • Create a new, unique Author (e.g., "IntegTest Author"). • Create a new Book (e.g., "IntegTest Book") and link it to "IntegTest Author". • Verify the book details page shows "IntegTest Author" as the author. • Navigate to Author Management and delete "IntegTest Author". • Navigate back to Book Management and view the details of "IntegTest Book".	• The Author is successfu deleted. • The Book "IntegTest B still exists in the system • The ‘authorId‘ field for "IntegTest Book" is han gracefully (e.g., set to n an empty value), as the model indicates it’s opt The book details page n longer displays the dele author’s name. • The system does not cr and remains in a consis state.

Continued on nex

Table 3.7 – continued from previous page

Test Case ID	Test Title / Scenario	Test Steps	Expected Results
TC_INT_003	Verify that deleting a User with active Borrowals does not violate data consistency.	• **As Librarian:** Login. • Ensure a specific Member (e.g., "MemberToDelete") has at least one active borrowal record. If not, create one. • Navigate to User Management. • Attempt to delete the user "MemberToDelete".	• **Expected (Option A Deletion Blocked):** T system prevents the del and displays an error m indicating the user has borrowals that must be resolved first. • **Expected (Option B Soft Delete/Orphani** The user is deleted. Th 'memberId' in the Borr record remains but now points to a non-existent The borrowal record is visible to the Librarian, may indicate the user is deleted. • (The exact expected res depends on the implem business rule for referen integrity, which should consistent).
Scenario: State Transitions Across Modules			
TC_INT_004	Verify that the state of a Book (isAvailable) is correctly updated by actions in the Borrowal module.	• **Prerequisite:** A book "StateTest Book" exists and is available. A "StateTest Member" exists. • **As Member:** Login as "StateTest Member". Borrow "StateTest Book". • **As Librarian:** Login. Navigate to Book Management. View the status of "StateTest Book". • Try to create a new borrowal for "StateTest Book" for another member. • Navigate to Borrowal Management. Find the borrowal record for "StateTest Book" and update its status to "Returned". • Navigate back to Book Management and view the status of "StateTest Book" again.	• After the member borro the book, the book's 'isAvailable' status immediately changes to 'false'. • The Librarian cannot c a new borrowal for the now-unavailable book. UI should prevent this a • After the Librarian mar the borrowal as "Return the borrowal record's st field is updated. • The 'isAvailable' status "StateTest Book" in the module reverts to 'true' • The data remains consi between the Book and Borrowal modules throughout the workflo
Scenario: Role-Based Access Control (RBAC) Integration			

Continued on nex

Table 3.7 – continued from previous page

Test Case ID	Test Title / Scenario	Test Steps	Expected Results
TC_INT_005	Verify that user roles created in the User module correctly restrict access to features in other modules (e.g., Book, Author).	• **As Librarian:** Login. Create a new user "TestMember" with the role of Member (‘isAdmin=false‘). • Logout. • **As Member:** Login as "TestMember". • Attempt to access the User Management page via direct URL navigation (e.g., ‘/users‘). • Navigate to the Books list. • Check for the presence of "Add New Book", "Edit", or "Delete" buttons/options. • Using an API client (e.g., Postman), attempt to send a ‘POST‘ request to ‘/api/book/add‘ using the session/token of "TestMember".	• The Member login is successful. • Direct navigation to the ‘/users‘ URL is blocked the user is redirected to member dashboard or a error page. • The UI for the Books li does not show any administrative controls Edit, Delete). • The API request to ‘PO /api/book/add‘ fails wi authorization error (e.g Forbidden or 401 Unauthorized), demonstrating that the access control middlewa correctly uses the user's from the session.

3.2.3 System Testing

System testing evaluates the complete and fully integrated application from an end-to-end (E2E) perspective. This level of testing simulates real user scenarios to validate the system against the requirements defined in the SRS.

- **Objective:** To validate the complete system's compliance with its specified requirements.
- **Framework:** Cypress is used for E2E testing, automating user interactions in a real browser environment.
- **Methodology:** Test scripts simulate user workflows such as logging in, searching for a book, borrowing a book, and managing user accounts. Assertions are made on the UI to confirm that the application behaves as expected.

3.3 Dynamic Testing Types

Dynamic testing is performed while the code is executing. We have categorized our dynamic tests into functional and non-functional types to ensure comprehensive coverage.

3.3.1 Functional Testing

Functional testing verifies that the system performs its functions as specified in the SRS. Test cases are derived directly from the user stories and functional requirements.

3.3.1.1 Authentication and Authorization

Table 3.8: Authentication and Authorization Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: User Registration (by Librarian)			
TC_AUTH_REG_001	Successful new user (Member) registration by Librarian	• Login as Librarian. • Navigate to 'User Management' or 'Add User' section. • Fill in all required fields with valid and unique data for a new 'Member' (e.g., email, password, full name). • Submit the registration form.	• System accepts the data. • A success message "User registered successfully" (or similar) is displayed. • The new user appears in the list of users. • The new user can subsequently log in (to be tested in Login cases).
TC_AUTH_REG_002	Attempt to register a new user with an existing email	• Login as Librarian. • Navigate to 'Add User' section. • Fill in user details, using an email address that already exists in the system. • Submit the registration form.	• System rejects the registration. • An appropriate error message is displayed (e.g., "Email already exists"). • The user is not created.
TC_AUTH_REG_003	Attempt to register a new user with missing required fields	• Login as Librarian. • Navigate to 'Add User' section. • Fill in user details but leave one or more mandatory fields blank (e.g., password or email). • Submit the registration form.	• System rejects the registration. • An appropriate error message is displayed, indicating which fields are required. • The user is not created.
TC_AUTH_REG_004	Attempt to register a new user with invalid data format (e.g., email)	• Login as Librarian. • Navigate to 'Add User' section. • Fill in user details, providing an incorrectly formatted email address (e.g., "usertest.com"). • Submit the registration form.	• System rejects the registration. • An appropriate error message is displayed (e.g., "Invalid email format"). • The user is not created.
Sub-Section: User Login			

Continued on next page

Table 3.8 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_AUTH_LOGIN_001	Successful login with valid Librarian credentials	• Navigate to the login page. • Enter valid Librarian email. • Enter corresponding valid password. • Click the 'Login' button.	• User is authenticated successfully. • User is redirected to the Librarian dashboard/homepage. • Librarian-specific functionalities are visible/accessible.
TC_AUTH_LOGIN_002	Successful login with valid Member credentials	• Navigate to the login page. • Enter valid Member email. • Enter corresponding valid password. • Click the 'Login' button.	• User is authenticated successfully. • User is redirected to the Member dashboard/homepage. • Member-specific functionalities are visible/accessible.
TC_AUTH_LOGIN_003	Attempt login with invalid email	• Navigate to the login page. • Enter an email that does not exist. • Enter any password. • Click the 'Login' button.	• Login fails. • An appropriate error message is displayed (e.g., "Invalid email or password"). • User remains on the login page.
TC_AUTH_LOGIN_004	Attempt login with valid email but invalid password	• Navigate to the login page. • Enter a valid existing email. • Enter an incorrect password for that email. • Click the 'Login' button.	• Login fails. • An appropriate error message is displayed (e.g., "Invalid email or password"). • User remains on the login page.
TC_AUTH_LOGIN_005	Attempt login with empty email field	• Navigate to the login page. • Leave the email field empty. • Enter any password. • Click the 'Login' button.	• Login fails. • An appropriate error message is displayed (e.g., "Email is required"). • User remains on the login page.

Continued on next page

Table 3.8 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_AUTH_LOGIN_006	Attempt login with empty password field	• Navigate to the login page. • Enter a valid email. • Leave the password field empty. • Click the 'Login' button.	• Login fails. • An appropriate error message is displayed (e.g., "Password is required"). • User remains on the login page.
Sub-Section: User Logout			
TC_AUTH_LOGOUT_001	Successful logout for a logged-in Librarian	• Login as Librarian. • Click the 'Logout' button/link.	• User is logged out successfully. • User is redirected to the login page or public homepage. • Session is terminated (user cannot access protected pages without logging in again).
TC_AUTH_LOGOUT_002	Successful logout for a logged-in Member	• Login as Member. • Click the 'Logout' button/link.	• User is logged out successfully. • User is redirected to the login page or public homepage. • Session is terminated.
Sub-Section: Role-Based Access Control (RBAC)			
TC_AUTH_RBAC_001	Verify Librarian can access Librarian-specific features (e.g., User Management)	• Login as Librarian. • Attempt to navigate to 'User Management' section. • Attempt to navigate to 'Book Management' (add/edit/delete books).	• Librarian successfully accesses 'User Management' page. • Librarian can view and interact with user administration tools. • Librarian successfully accesses 'Book Management' page and can perform CRUD operations on books.

Continued on next page

Table 3.8 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_AUTH_RBAC_002	Verify Member cannot access Librarian-specific features (e.g., User Management)	• Login as Member. • Check if 'User Management' link/button is visible. • Attempt to navigate directly to the User Management URL (if known). • Check if 'Add Book' or 'Edit Book' options are available.	• 'User Management' link/button is not visible or is disabled. • Direct navigation to User Management URL redirects to an error page or member dashboard. • 'Add Book'/'Edit Book' options are not available. Access is denied.
TC_AUTH_RBAC_003	Verify Member can access Member-specific features (e.g., Search Books, View Borrowal History)	• Login as Member. • Attempt to navigate to 'Search Books' page. • Attempt to view 'My Borrowal History'.	• Member successfully accesses 'Search Books' page and can search. • Member successfully accesses 'My Borrowal History' and can view their own history.
TC_AUTH_RBAC_004	Verify guest (not logged in) user access to public pages and restriction from protected pages	• Ensure no user is logged in (clear session/cookies or use incognito window). • Attempt to navigate to the login page. • Attempt to navigate to a public book search page (if applicable). • Attempt to navigate to 'User Management' URL. • Attempt to navigate to 'My Borrowal History' URL.	• Login page is accessible. • Public book search page is accessible (if such a feature exists for guests). • Access to 'User Management' URL is denied, likely redirecting to login page. • Access to 'My Borrowal History' URL is denied, likely redirecting to login page.

3.3.1.2 Entity Management Testing

Table 3.9: Entity Management Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: Book Management (SRS 3.3)			
TC_BOOK_CREATE_001	Successful new book creation by Librarian	• Login as Librarian. • Navigate to 'Book Management'. • Click 'New Book'. • Fill all required fields (Name, ISBN) with valid, unique data. Select existing Author and Genre. • Submit the form.	• System accepts the data. • A success message is displayed. • The new book appears in the book list.
TC_BOOK_CREATE_002	Attempt to create a book with missing required fields	• Login as Librarian. • Navigate to 'Book Management'. • Click 'New Book'. • Leave the 'Name' or 'ISBN' field blank. • Submit the form.	• System rejects the creation. • An error message indicating required fields is displayed. • The book is not created.
TC_BOOK_CREATE_003	(RBAC) Verify Member cannot create a book	• Login as a Member. • Navigate to 'Book Management'.	• The 'New Book' button/option is not visible or is disabled. • Direct access to the create book API endpoint (e.g., POST /api/book/add) should be denied.
TC_BOOK_READ_001	Verify all users can view the list of books	• Login as Librarian. Navigate to 'Book Management'. • Logout. Login as Member. Navigate to 'Book Management'.	• Both Librarian and Member can successfully view the list of all books.

Continued on next page

Table 3.9 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_BOOK_UPDATE_001	Successful book update by Librarian	• Login as Librarian. • Navigate to 'Book Management'. • Select a book and choose to edit it. • Modify a field (e.g., the summary). • Save the changes.	• System accepts the changes. • A success message is displayed. • The book list/details reflect the updated information.
TC_BOOK_DELETE_001	Successful book deletion by Librarian	• Pre-condition: Create a book that has no associated borrowals. • Login as Librarian. • Navigate to 'Book Management'. • Select the book and click 'Delete'. • Confirm the deletion.	• System accepts the deletion. • A success message is displayed. • The book is removed from the book list.
TC_BOOK_DELETE_002	(Referential Integrity) Attempt to delete a book that is currently borrowed	• Pre-condition: A book exists and is part of an active borrowal record. • Login as Librarian. • Navigate to 'Book Management'. • Attempt to delete the borrowed book.	• System prevents the deletion. • An appropriate error message is displayed (e.g., "Cannot delete a book with active borrowals"). • The book is not deleted.
Sub-Section: Author Management (SRS 3.4)			

Continued on next page

Table 3.9 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_AUTHOR_CREATE_001	Successful new author creation by Librarian	• Login as Librarian. • Navigate to 'Author Management'. • Click 'New Author'. • Fill the required 'Name' field with valid data. • Submit the form.	• A success message is displayed. • The new author appears in the authors list.
TC_AUTHOR_DELETE_001	(Referential Integrity) Attempt to delete an author linked to a book	• Pre-condition: An author is assigned to at least one book in the system. • Login as Librarian. • Navigate to 'Author Management'. • Attempt to delete the author.	• System prevents the deletion. • An error message is displayed (e.g., "Cannot delete author assigned to existing books"). • The author is not deleted.
Sub-Section: Genre Management (SRS 3.5)			
TC_GENRE_CREATE_001	Successful new genre creation by Librarian	• Login as Librarian. • Navigate to 'Genre Management'. • Click 'New Genre'. • Fill the required 'Name' and 'Description' fields with valid data. • Submit the form.	• A success message is displayed. • The new genre appears in the genres list.

Continued on next page

Table 3.9 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_GENRE_DELETE_001	(Referential Integrity) Attempt to delete a genre linked to a book	• Pre-condition: A genre is assigned to at least one book in the system. • Login as Librarian. • Navigate to 'Genre Management'. • Attempt to delete the genre.	• System prevents the deletion. • An error message is displayed (e.g., "Cannot delete genre assigned to existing books"). • The genre is not deleted.
Sub-Section: User Management (SRS 3.7)			
TC_USER_CREATE_001	Successful new user (Member) creation by Librarian	• Login as Librarian. • Navigate to 'User Management'. • Click 'New User'. • Fill required fields (Name, Email, Password) with valid, unique data. Set 'isAdmin' flag to false. • Submit the form.	• A success message is displayed. • The new user appears in the user list.
TC_USER_CREATE_002	Attempt to create a user with an existing email	• Login as Librarian. • Navigate to 'User Management'. • Attempt to create a new user with an email address that is already in use.	• System rejects the creation. • An error message like "User already exists" is displayed.
TC_USER_READ_001	(RBAC) Verify Member cannot access User Management	• Login as a Member.	• 'User Management' link/feature is not visible or is disabled. • Direct navigation to the user management URL is denied.

Continued on next page

Table 3.9 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_USER_DELETE_001	(Referential Integrity) Attempt to delete a member with active borrowals	• Pre-condition: A member has one or more active (not returned) borrowal records. • Login as Librarian. • Navigate to 'User Management'. • Attempt to delete the member.	• System prevents the deletion. • An error message is displayed (e.g., "Cannot delete member with active borrowals"). • The user is not deleted.
Sub-Section: Borrowal Management (SRS 3.6)			
TC_BORROW_CREATE_001	Successful borrowal creation by Member	• Pre-condition: A book is available (isAvailable=true). • Login as Member. • Navigate to 'Borrowal Management'. • Click 'New Borrowal' and select an available book. • Submit the form.	• A new borrowal record is created with 'Borrowed' status. • A success message is displayed. • The book's availability status is updated to 'false'.
TC_BORROW_CREATE_002	Attempt to borrow an unavailable book	• Pre-condition: A book is unavailable (isAvailable=false). • Login as Member. • Attempt to create a borrowal for the unavailable book.	• The system prevents the borrowal. • An appropriate error message is displayed (e.g., "Book is not available").
TC_BORROW_READ_001	Verify Member can only see their own borrowal history	• Pre-condition: Multiple members have borrowal records. • Login as Member 1. • Navigate to 'Borrowal Management'.	• The list only displays borrowals where the memberId matches Member 1's ID. • Borrowals for other members are not visible.

Continued on next page

Table 3.9 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_BORROW_UPDATE_001	Successful borrowal update by Librarian (mark as returned)	• Pre-condition: An active borrowal exists. • Login as Librarian. • Navigate to 'Borrowal Management'. • Select the active borrowal and edit it. • Change the status to 'Returned'. • Save the changes.	• The borrowal record status is updated. • The associated book's 'isAvailable' status should ideally be updated to 'true'. • A success message is displayed.
Sub-Section: Review Management (SRS 3.8)			
TC_REVIEW_CREATE_001	Successful review creation by Member	• Pre-condition: Based on assumed member capabilities if UI were present. • Login as Member. • Navigate to a specific book's detail page. • Submit a new review with a star rating and text.	• A success message is displayed. • The new review is visible on the book's page.
TC_REVIEW_CREATE_002	Attempt to create a review with invalid data	• Login as Member. • Attempt to submit a review with a required field missing (e.g., star rating) or an invalid value (e.g., 6 stars).	• System rejects the submission. • An appropriate error message is displayed.

Continued on next page

Table 3.9 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_REVIEW_DELETE_001	Successful review deletion by Librarian	• Pre-condition: A review exists for a book. • Login as Librarian. • Navigate to the book or a review management section. • Select a review and delete it.	• The review is successfully deleted. • The review is no longer visible.

3.3.1.3 Use Case Testing

Table 3.10: Use Case Test Cases

Test Case ID	Test Title / Use Case	Test Steps	Expected Results
Sub-Section: UC-002: Add New Book (Librarian)			
TC_UC_BOOK_001	Successful addition of a new book (Main Flow)	• As a logged-in Librarian, navigate to the Book Management page (‘/books‘). • Click the "New Book" button. • Fill in all required fields (Name, ISBN) with valid data. • Select a valid Author and Genre from the provided lists. • Ensure "Is Available" is set to true. • Submit the form.	• The system creates a new book record via ‘POST /api/book/add‘. • A success message is displayed (e.g., "Book added"). • The "Add Book" form closes. • The new book appears in the list on the Book Management page.

Continued on next page

Table 3.10 – continued from previous page

Test Case ID	Test Title / Use Case	Test Steps	Expected Results
TC_UC_BOOK_002	Attempt to add a book with missing required fields (Alternative Flow A1)	• As a logged-in Librarian, navigate to the "Add Book" form. • Fill in the form but leave a required field (e.g., Name) blank. • Submit the form.	• The system rejects the submission due to a validation error. • An error message is displayed, indicating the required field is missing. • The form remains open for correction. • The book is not added to the database.
TC_UC_BOOK_003	Attempt to add a book that fails backend validation (Alternative Flow A2)	• As a logged-in Librarian, navigate to the "Add Book" form. • Fill in all fields with data that is syntactically valid on the client-side but might trigger a server error (e.g., a duplicate ISBN if the server enforces uniqueness). • Submit the form.	• The system displays a generic error message from the API (e.g., "Something went wrong, please try again"). • The book is not added to the database.
Sub-Section: UC-003: Borrow a Book (Member)			
TC_UC_BORROW_001	Successful borrowing of an available book (Main Flow)	• As a logged-in Member, navigate to the Borrowal Management page (‘/borrowals‘). • Click the "New Borrowal" button. • Select a book that is marked as available from the list. • Verify that the Member field is correctly auto-populated. • Submit the form.	• A new borrowal record is created with "Borrowed" status via ‘POST /api/borrowal/add‘. • The availability of the book is updated to ‘false‘. • A success message is displayed. • The member’s borrowal list is updated to show the new borrowal.

Continued on next page

Table 3.10 – continued from previous page

Test Case ID	Test Title / Use Case	Test Steps	Expected Results
TC_UC_BORROW_002	Attempt to borrow an unavailable book (Alternative Flow A1)	• As a logged-in Member, navigate to the "Add Borrowal" form. • Attempt to select a book that is not available (isAvailable=false).	• The UI should ideally prevent the selection of an unavailable book. • If selection is possible, the system should prevent submission or display an error message (e.g., "Book is not available") upon submission attempt. • No borrowal record is created.
TC_UC_BORROW_003	Verify correct data population in borrowal form	• As a logged-in Member, open the "Add Borrowal" form. • Observe the date fields.	• The Member field is correctly auto-populated with the current user's ID. • The Borrowed Date is auto-populated to the current date. • The Due Date is auto-populated to a future date (e.g., 14 days from current). • The initial status is correctly set (e.g., "Borrowed").
Sub-Section: UC-005: View Borrowal History (Member)			

Continued on next page

Table 3.10 – continued from previous page

Test Case ID	Test Title / Use Case	Test Steps	Expected Results
TC_UC_HISTORY_001	View non-empty borrowal history (Main Flow)	• Log in as a Member who has previously borrowed one or more books. • Navigate to the Borrowal Management page (‘/borrowals‘).	• The system fetches all borrowals and the client filters them. • A list/table is displayed showing *only* the records where the ‘memberId‘ matches the logged-in Member’s ID. • The displayed data (Book Name, Borrowed Date, Due Date, Status) is accurate for each record.
TC_UC_HISTORY_002	View empty borrowal history (Alternative Flow A1)	• Log in as a new Member with no borrowal records. • Navigate to the Borrowal Management page (‘/borrowals‘).	• The system displays a message indicating there is no borrowal history (e.g., "No borrowals found").
TC_UC_HISTORY_003	Verify role-based view of borrowal history	• Prerequisite: There are borrowals from multiple members in the system. • Log in as a Librarian and navigate to the Borrowal Management page (‘/borrowals‘). • Log out and log in as a Member (who has at least one borrowal) and navigate to the same page.	• As the Librarian, all borrowal records for all members are visible. • As the Member, only their own borrowal history is visible.
Note: UC-001 (User Login) and UC-004 (Register New User) are detailed in the Authentication & Authorization test plan.			

3.3.1.4 State Transition Testing

Table 3.11: State Transition Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Entity: Borrowal Record (Based on 'status' attribute)			
TC_STATE_BORROW_001	Valid Transition: (None) to 'Borrowed'	• Login as a Member. • Navigate to the Borrowal Management page. • Create a new borrowal record for an available book. • Submit the form.	• A new borrowal record is successfully created. • The status of the new record is set to "Borrowed". • A success message is displayed.
TC_STATE_BORROW_002	Valid Transition: 'Borrowed' to 'Returned'	• Precondition: A book is in 'Borrowed' status by a member. • Login as Librarian. • Navigate to Borrowal Management. • Locate the 'Borrowed' record. • Update the record's status to 'Returned' before its due date.	• The system accepts the update. • The borrowal record's status changes from "Borrowed" to "Returned". • A success message is displayed.
TC_STATE_BORROW_003	Valid Transition: 'Borrowed' to 'Overdue'	• Precondition: A book is in 'Borrowed' status by a member. • Manually adjust system time or the record's due date to be in the past. • As a Librarian, view the borrowal record. (Or trigger a system check for overdue items).	• The borrowal record's status automatically (or upon view) changes from "Borrowed" to "Overdue". • The status is displayed correctly in the UI.
TC_STATE_BORROW_004	Valid Transition: 'Overdue' to 'Returned'	• Precondition: A book is in 'Overdue' status. • Login as Librarian. • Locate the 'Overdue' borrowal record. • Update the record's status to 'Returned'.	• The system accepts the update. • The borrowal record's status changes from "Overdue" to "Returned". • A success message is displayed.

Continued on next page

Table 3.11 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_STATE_BORROW_005	Invalid Transition: 'Returned' to 'Borrowed'	• Precondition: A borrowal record has a 'Returned' status. • As a Librarian, attempt to edit the same record and change its status back to 'Borrowed'.	• The system should reject the change. • The UI should not provide an option for this transition, or it should result in an error if attempted via API. • An appropriate error message is displayed (e.g., "Cannot revert a returned item").
Entity: Book (Based on 'isAvailable' attribute)			
TC_STATE_BOOK_001	Valid Transition: 'Available' to 'Unavailable'	• Precondition: A book exists with 'isAvailable' status set to true. • As a Member, create a new borrowal for that specific book. • After the borrowal is confirmed, view the details of that book.	• The borrowal transaction is successful. • The 'isAvailable' status of the book is updated to false. • The book's listing correctly shows it as unavailable.
TC_STATE_BOOK_002	Valid Transition: 'Unavailable' to 'Available'	• Precondition: A book has 'isAvailable' status set to false due to being borrowed. • As a Librarian, locate the corresponding borrowal record. • Update the borrowal record status to 'Returned'. • View the details of the book again.	• The 'isAvailable' status of the book is updated to true. • The book's listing correctly shows it as available for borrowing.

Continued on next page

Table 3.11 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_STATE_BOOK_003	Invalid Transition: Attempt to borrow an 'Unavailable' book	• Precondition: A book has 'isAvailable' status set to false. • As a Member, attempt to create a new borrowal for that specific book.	• The system prevents the borrowal request from being submitted. • An appropriate error message is displayed (e.g., "Book is currently unavailable"). • No new borrowal record is created.
Entity: User Session (Based on login/logout actions)			
TC_STATE_SESSION_001	Valid Transition: 'Logged-Out' to 'Logged-In'	• Start from a logged-out state (e.g., in an incognito window). • Navigate to the login page. • Enter valid credentials for a 'Member'. • Click 'Login'.	• User session is established. • User is redirected to the member landing page (e.g., /books). • User can access member-specific pages.
TC_STATE_SESSION_002	Valid Transition: 'Logged-In' to 'Logged-Out'	• Precondition: A user is logged in. • Click the 'Logout' button/link.	• The user's session is terminated. • User is redirected to the login page.
TC_STATE_SESSION_003	Invalid Transition: Accessing protected page when 'Logged-Out'	• Start from a logged-out state. • Attempt to directly navigate to a protected URL (e.g., /borrowals, /users).	• Access to the protected page is denied. • User is redirected to the login page.
TC_STATE_SESSION_004	State Persistence: Verify session remains 'Logged-In' after page refresh	• Login as any user. • Navigate to any page other than the login page. • Refresh the web browser page.	• The user remains logged in. • The user stays on the same page. • No redirection to the login page occurs.

3.3.1.5 Specific Feature Testing

Table 3.12: Specific Feature Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: Dashboard (Librarian)			
TC_DASH_001	Verify Librarian can access the Dashboard	• Login as Librarian. • Navigate to the Dashboard.	• User is successfully redirected to the Librarian dashboard at '/dashboard'. • Dashboard displays overview statistics and charts.
TC_DASH_002	Verify Member cannot access the Dashboard	• Login as a 'Member' user. • Attempt to navigate directly to the '/dashboard' URL.	• Access is denied. • User is redirected to the member landing page (e.g., '/books') or an error page.
Sub-Section: Book Management (CRUD)			
TC_BOOK_ADD_001	Successful new book creation by Librarian	• Login as Librarian. • Navigate to 'Book Management' (/books). • Click "New Book" button. • Fill in all required fields (Name, ISBN) with valid data. • Select existing Author and Genre. • Submit the form.	• A success message is displayed. • The new book record is created and appears in the book list.
TC_BOOK_ADD_002	Attempt to create a book with missing required fields	• Login as Librarian. • Navigate to 'Book Management'. • Open the "Add Book" form. • Leave a required field (e.g., Name) blank. • Submit the form.	• The system rejects the submission. • An error message indicating the required field is missing is displayed. • The book is not added to the database.
TC_BOOK_VIEW_001	Verify Member can view list of books and book details	• Login as Member. • Navigate to 'Book Management' (/books). • Click on a specific book in the list.	• Member can successfully view the list of all books. • Member can successfully view the details of a specific book.

Continued on next page

Table 3.12 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_BOOK_UPD_001	Successful update of a book's details by Librarian	• Login as Librarian. • Navigate to 'Book Management'. • Select a book to edit. • Change a field (e.g., update the summary). • Save the changes.	• A success message is displayed. • The updated information is reflected correctly in the book's detail view.
TC_BOOK_DEL_001	Successful deletion of a book by Librarian	• Login as Librarian. • Navigate to 'Book Management'. • Select a book to delete. • Confirm the deletion in the dialog.	• A success message is displayed. • The book is removed from the book list.
TC_BOOK_ACCESS_001	Verify Member cannot access book CRUD operations	• Login as Member. • Navigate to 'Book Management'.	• "Add Book", "Edit Book", and "Delete Book" UI elements are not visible or are disabled. • Direct access to corresponding API endpoints is denied.
Sub-Section: Borrowal Management			
TC_BORW_ADD_001	Successful borrowal request by Member for an available book	• Login as Member. • Navigate to 'Borrowal Management' (/borrowals). • Click "New Borrowal". • Select a book that is marked as available. • Submit the form.	• A success message is displayed. • A new borrowal record is created with 'Borrowed' status and correct dates. • The new record appears in the member's borrowal history.
TC_BORW_ADD_002	Attempt to borrow a book that is not available	• Login as Member. • Navigate to 'Borrowal Management'. • Attempt to create a new borrowal for a book that is not available.	• The UI should clearly indicate the book is not available. • The system prevents the submission or displays a clear error message.

Continued on next page

Table 3.12 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_BORW_VIEW_001	Verify Member can only view their own borrowal history	• Login as a Member who has borrowal records. • Navigate to 'Borrowal Management' (/borrowals).	• The list displays only the borrowals associated with the logged-in member's ID. • Records of other members are not visible.
TC_BORW_VIEW_002	Verify Librarian can view all borrowal records	• Login as Librarian. • Navigate to 'Borrowal Management'.	• A list of all borrowal records for all members is displayed.
TC_BORW_UPD_001	Successful update of borrowal status by Librarian	• Login as Librarian. • Navigate to 'Borrowal Management'. • Select an existing 'Borrowed' record. • Update its status to 'Returned'. • Save the change.	• A success message is displayed. • The record's status is correctly updated in the list.
Sub-Section: User Management (by Librarian)			
TC_USER_VIEW_001	Verify Librarian can view list of all users	• Login as Librarian. • Navigate to 'User Management' (/users).	• A list of all users (both Librarians and Members) is displayed correctly.
TC_USER_UPD_001	Successful update of a user's details by Librarian	• Login as Librarian. • Navigate to 'User Management'. • Select a user to edit. • Update a non-critical field (e.g., Phone number). • Save the changes.	• A success message is displayed. • The user's details are correctly updated.
TC_USER_DEL_001	Successful deletion of a user by Librarian	• Login as Librarian. • Navigate to 'User Management'. • Select a user account to delete. • Confirm the deletion.	• A success message is displayed. • The user is removed from the list of users. • The deleted user can no longer log in.
Sub-Section: Review Management (Assumed Admin UI)			

Continued on next page

Table 3.12 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_REV_ADD_001	Successful addition of a review by Librarian (Admin action)	• Login as Librarian. • Navigate to a book's detail page. • Use an administrative function to add a review for the book. • Enter a star rating and review text. • Submit the review.	• A success message is displayed. • The new review appears in the list of reviews for that book.
TC_REV_VIEW_001	Verify any user can view reviews for a book	• As any logged-in user (or guest, if public). • Navigate to the detail page of a book that has reviews.	• The list of reviews for the book is displayed correctly.
TC_REV_DEL_001	Successful deletion of a review by Librarian	• Login as Librarian. • Navigate to a book with reviews. • Select a specific review to delete. • Confirm the deletion.	• A success message is displayed. • The review is removed from the book's list of reviews.

3.3.2 API Testing

API testing is performed to ensure the reliability, functionality, performance, and security of the application's API layer.

Table 3.13: API Test Cases

Test Case ID	Objective	API Endpoint & Method	Request Body / Parameters Expected Results
Sub-Section: Authentication API (‘/api/auth‘)			
			Continued on next p

Table 3.13 – continued from previous page

Test Case ID	Objective	API Endpoint & Method	Request Body / Parameters Expected Results
TC_API_AUTH_001	Successful Librarian Login	**POST** '/api/auth/login'	**Request Body (JSON):** `{ "email": "librarian@example.com", "password": "valid_password" }` **Expected Result:** • **Status Code:** 200 OK • **Response Body:** Contains user object with 'isAdmin: tr and an authentication token/session cookie.
TC_API_AUTH_002	Successful Member Login	**POST** '/api/auth/login'	**Request Body (JSON):** `{ "email": "member@example.com", "password": "valid_password" }` **Expected Result:** • **Status Code:** 200 OK • **Response Body:** Contains user object with 'isAdmin: fa and an authentication token/session cookie.
TC_API_AUTH_003	Login with invalid password	**POST** '/api/auth/login'	**Request Body (JSON):** `{ "email": "librarian@example.com", "password": "invalid_password" }` **Expected Result:** • **Status Code:** 401 Unauthorized (or similar) • **Response Body:** Contains appropriate error message.

Continued on next p

Table 3.13 – continued from previous page

Test Case ID	Objective	API Endpoint & Method	Request Body / Parameters Expected Results
TC_API_AUTH_004	Librarian registers a new member	**POST** '/api/auth/register' *(Requires Librarian auth)*	**Request Body (JSON):** `{ "name": "New Member", "email": "new.member@example.com", "password": "strong_password", "isAdmin": false }` **Expected Result:** • **Status Code:** 201 Created • **Response Body:** Confirmat message. The user record is created in the database with hashed password.
TC_API_AUTH_005	Attempt to register with an existing email	**POST** '/api/auth/register' *(Requires Librarian auth)*	**Request Body (JSON):** `{ "name": "Another Member", "email": "member@example.com", "password": "another_password", "isAdmin": false }` **Expected Result:** • **Status Code:** 400 Bad Requ (or similar) • **Response Body:** Error message indicating the user already exists.
TC_API_AUTH_006	Member attempts to access user registration endpoint	**POST** '/api/auth/register' *(Requires Member auth)*	**Request Body (JSON):** (any valid user data) **Expected Result:** • **Status Code:** 403 Forbidde • **Response Body:** Error message indicating insufficien permissions, as only librarian can register users.
TC_API_AUTH_007	Successful Logout	**GET** '/api/auth/logout' *(Requires any user auth)*	**Request Body:** N/A **Expected Result:** • **Status Code:** 200 OK (or redirect) • The user's session is terminat Subsequent requests to protected endpoints should fa
Sub-Section: Book Management API ('/api/book')			

Continued on next p

Table 3.13 – continued from previous page

Test Case ID	Objective	API Endpoint & Method	Request Body / Parameters Expected Results
TC_API_BOOK_001	Librarian adds a new book	**POST** ‘/api/book/add‘ *(Requires Librarian auth)*	**Request Body (JSON):** `{ "name": "The Art of Software Testing", "isbn": "978-1118031534", "isAvailable": true }` **Expected Result:** • **Status Code:** 201 Created • **Response Body:** The newly created book object.
TC_API_BOOK_002	Member attempts to add a new book	**POST** ‘/api/book/add‘ *(Requires Member auth)*	**Request Body (JSON):** (any valid book data) **Expected Result:** • **Status Code:** 403 Forbidden • **Response Body:** Error message indicating insufficien permissions. Librarians have exclusive add rights.
TC_API_BOOK_003	Any user gets a list of all books	**GET** ‘/api/book/getAll‘ *(Public or Member auth)*	**Request Body:** N/A **Expected Result:** • **Status Code:** 200 OK • **Response Body:** An array book objects.
TC_API_BOOK_004	Librarian deletes a book	**DELETE** ‘/api/book/delete/id‘ *(Requires Librarian auth)*	**Parameters:** URL parameter ‘i of an existing book. **Expected Result:** • **Status Code:** 200 OK (or 2 No Content) • **Response Body:** Success message. The book is remove from the database.
TC_API_BOOK_005	Unauthenticated user attempts to delete a book	**DELETE** ‘/api/book/delete/id‘ *(No auth)*	**Parameters:** URL parameter ‘i of an existing book. **Expected Result:** • **Status Code:** 401 Unauthorized • **Response Body:** Error message indicating authentication is required.
Sub-Section: Borrowal Management API (‘/api/borrowal‘)			

Continued on next p

Table 3.13 – continued from previous page

Test Case ID	Objective	API Endpoint & Method	Request Body / Parameters Expected Results
TC_API_BORROW_001	Member borrows an available book for themselves	**POST** ‘/api/borrowal/add‘ *(Requires Member auth)*	**Request Body (JSON):** `{ "bookId": "valid_book_id", "memberId": "current_user_id" }` **Expected Result:** • **Status Code:** 201 Created • **Response Body:** The new borrowal record. The book’s availability may be updated.
TC_API_BORROW_002	Member attempts to borrow a book for another member	**POST** ‘/api/borrowal/add‘ *(Requires Member auth)*	**Request Body (JSON):** `{ "bookId": "valid_book_id", "memberId": "another_user_id" }` **Expected Result:** • **Status Code:** 403 Forbidden • **Response Body:** Error message. The API must enfo that the ‘memberId‘ in the request matches the authenticated user’s ID.
TC_API_BORROW_003	Librarian updates a borrowal status (e.g., to ’Returned’)	**PUT** ‘/api/borrowal/update/id‘ *(Requires Librarian auth)*	**Parameters:** URL parameter ‘ of an existing borrowal. **Request Body (JSON):** `{ "status": "Returned" }` **Expected Result:** • **Status Code:** 200 OK • **Response Body:** The upda borrowal object.
TC_API_BORROW_004	Member attempts to view all borrowals in the system	**GET** ‘/api/borrowal/getAll‘ *(Requires Member auth)*	**Request Body:** N/A **Expected Result:** • **Status Code:** 200 OK • **Response Body:** An array containing **only** the borrowal records for the authenticated member. The API should not leak other users’ data.
Sub-Section: Security & Input Validation			

Continued on next p

Table 3.13 – continued from previous page

Test Case ID	Objective	API Endpoint & Method	Request Body / Parameters Expected Results
TC_API_SEC_001	Verify password hash is not returned on user lookup	**GET** ‘/api/user/get/id‘ *(Requires Librarian auth)*	**Parameters:** URL parameter ‘i for any user. **Expected Result:** • **Status Code:** 200 OK • **Response Body:** The user object **must not** contain the ‘hash‘ or ‘salt‘ fields.
TC_API_SEC_002	Attempt basic SQL Injection (or NoSQL equivalent)	**POST** ‘/api/auth/login‘	**Request Body (JSON):** `{ "email": "' OR '1'='1", "password": "' OR '1'='1" }` **Expected Result:** • **Status Code:** 401 Unauthorized (or 400 Bad Request) • **Response Body:** Normal lo failure message. The system must not be compromised. T validates server-side input validation.

3.3.3 White-Box Testing

White-box testing examines the internal logic and structure of the code. Code coverage is a key metric used to evaluate the thoroughness of our white-box testing efforts.

- **Objective:** To ensure that internal paths in the code are exercised and that the internal logic is sound.
- **Methodology:** We use Jest’s built-in coverage reporting tools, which are based on Istanbul. This helps us identify untested parts of the codebase.

3.4 Test Strategy and Coverage Criteria

3.4.1 Coverage Types Implemented

3.4.1.1 Statement Coverage

- **Target**: 90% statement coverage
- **Measurement**: Every executable statement tested at least once
- **Implementation**: Jest coverage reports track statement execution

3.4.1.2 Branch Coverage

- **Target**: 85% branch coverage
- **Measurement**: All conditional branches (if/else, switch cases) tested

- **Implementation**: Test cases cover both true and false conditions

3.4.1.3 Condition Coverage

- **Target**: 80% condition coverage
- **Measurement**: Individual conditions in complex boolean expressions tested
- **Implementation**: Separate test cases for each condition combination

3.4.1.4 Path Coverage

- **Target**: 75% path coverage for critical functions
- **Measurement**: Independent execution paths through functions tested
- **Implementation**: Test cases designed to exercise different execution paths

3.4.2 Complexity Analysis

3.4.2.1 Cyclomatic Complexity Assessment

Critical functions analyzed for complexity:

- **authController.registerUser**: Complexity = 8 (High)
- **authController.loginUser**: Complexity = 7 (Medium-High)
- **bookController.addBook**: Complexity = 9 (High)
- **User.setPassword**: Complexity = 2 (Low)
- **User.isValidPassword**: Complexity = 2 (Low)

3.4.2.2 Complexity Reduction Recommendations

1. Break down high-complexity functions into smaller, focused functions
2. Extract validation logic into separate utility functions
3. Implement strategy pattern for different authentication methods
4. Use middleware for common validation tasks

3.5 Server-Side White-Box Testing

3.5.1 Model Testing

3.5.1.1 User Model (TC_WB_USER_001 - TC_WB_USER_017)

File: `server/__tests__/unit/models/user.test.js`
Coverage Areas:

- Schema validation for required fields (name, email, isAdmin, photoUrl)
- Optional field validation (dob, phone)
- Password hashing logic using crypto.pbkdf2Sync

- Password validation logic with salt comparison
- Error handling for crypto operations
- Integration testing of complete password flow

Key Test Cases:

Test ID	Description	Coverage Type
TC_WB_USER_005	Should require name field	Statement
TC_WB_USER_006	Should generate random salt using crypto.randomBytes	Branch
TC_WB_USER_007	Should hash password with correct parameters	Path
TC_WB_USER_013	Should return true when hashes match	Condition
TC_WB_USER_015	Complete password set and validation flow	Integration
TC_WB_USER_016	Should handle crypto errors gracefully	Error Path

3.5.1.2 Book Model (TC_WB_BOOK_MODEL_001 - TC_WB_BOOK_MODEL_026)

File: `server/__tests__/unit/models/book.test.js`

Coverage Areas:

- Schema validation for required fields (name, isbn, isAvailable)
- Optional field validation (authorId, genreId, summary, photoUrl)
- Data type validation (String, Boolean, ObjectId)
- Edge cases (empty strings, null values, unicode characters)
- Mongoose integration validation

3.5.2 Controller Testing

3.5.3 Authentication Controller (TC_WB_AUTH_001 - TC_WB_AUTH_017)

File: `server/__tests__/unit/controllers/authController.test.js`

Coverage Areas:

- Input validation logic flow (email → password → name → password strength)
- Database interaction error handling
- User existence checking logic
- Password validation integration
- Passport authentication flow
- Session management error handling

Validation Flow Analysis:

```
// White-box testing reveals exact validation order:
1. if (!req.body.email) -> return emailRequired error
2. if (!req.body.password) -> return passwordRequired error
3. if (!req.body.name) -> return nameRequired error
4. if (req.body.password.length < 8) -> return weakPassword error
5. User.findOne() -> check existing user
6. Create new user if validation passes
```

Listing 3.11: Authentication Validation Flow

3.5.3.1 Book Controller (TC_WB_BOOK_001 - TC_WB_BOOK_008)

File: `server/__tests__/unit/controllers/bookController.test.js`
Coverage Areas:

- ObjectId validation using mongoose.Types.ObjectId.isValid()
- Input validation sequence (title → authorId → genreId → isbn)
- ISBN duplication checking logic
- Database aggregation pipeline testing
- Error handling for database operations

3.5.4 Utility Testing

3.5.4.1 Error Messages Utility (TC_WB_UTIL_001 - TC_WB_UTIL_026)

File: `server/__tests__/unit/utils/errorMessages.test.js`
Coverage Areas:

- Error message structure validation
- getErrorMessage function logic with fallback handling
- Category and key validation
- Edge cases (null, undefined, empty strings)
- Message content consistency validation

Function Logic Analysis:

```
function getErrorMessage(category, key, fallback) {
  // White-box testing covers all paths:
  if (errorMessages[category] && errorMessages[category][key]) {
    return errorMessages[category][key];  // Path 1: Valid category and key
  }
  return fallback || errorMessages.general.serverError;  // Path 2:
    Fallback
}
```

Listing 3.12: getErrorMessage Logic Flow

3.6 Client-Side White-Box Testing

3.6.1 Utility Function Testing

3.6.1.1 Format Number Utility (TC_WB_CLIENT_001 - TC_WB_CLIENT_040)

File: `client/src/__tests__/unit/utils/formatNumber.test.js`
Coverage Areas:

- Number formatting functions (fNumber, fCurrency, fPercent, fShortenNumber, fData)
- Internal result function logic for suffix removal
- Edge cases (NaN, Infinity, very large/small numbers)

- Error handling for invalid inputs
- Numeral.js integration testing

Internal Logic Analysis:

```
export function fCurrency(number) {
  // White-box testing reveals conditional logic:
  const format = number ? numeral(number).format('$0,0.00') : '';  // Branch 1
  return result(format, '.00');  // Always calls result function
}

function result(format, key = '.00') {
  const isInteger = format.includes(key);  // Condition testing
  return isInteger ? format.replace(key, '') : format;  // Branch coverage
}
```

Listing 3.13: Currency Formatting Logic

3.7 Code Coverage Analysis

3.7.1 Coverage Metrics

3.7.1.1 Server-Side Coverage

Component	Statements	Branches	Functions	Lines
Models	95%	90%	100%	95%
Controllers	88%	82%	95%	88%
Utilities	100%	95%	100%	100%
Overall	**91%**	**86%**	**97%**	**91%**

Table 3.15: Server-Side Code Coverage Results

3.7.1.2 Client-Side Coverage

Component	Statements	Branches	Functions	Lines
Utilities	98%	95%	100%	98%
Components	75%	70%	85%	75%
Overall	**82%**	**78%**	**90%**	**82%**

Table 3.16: Client-Side Code Coverage Results

3.7.2 Coverage Gaps and Recommendations

3.7.2.1 Identified Gaps

1. **Error Handling Paths**: Some error scenarios in controllers not fully covered
2. **Async Operations**: Complex async/await patterns need additional testing
3. **React Components**: Event handlers and lifecycle methods require more coverage
4. **Integration Points**: Database connection error scenarios

3.7.2.2 Improvement Recommendations

1. Implement additional error injection tests
2. Add more comprehensive async operation testing
3. Increase React component interaction testing
4. Implement database connection failure simulation

3.8 Static Analysis Results

3.8.1 ESLint Analysis

3.8.1.1 Code Quality Metrics

- **Total Issues**: 23 warnings, 0 errors
- **Complexity Issues**: 3 functions exceed recommended complexity
- **Code Smells**: 5 instances of code duplication identified
- **Best Practices**: 15 minor violations (unused variables, console.log statements)

3.8.1.2 Critical Issues Addressed

1. Reduced cyclomatic complexity in authController.registerUser
2. Extracted common validation logic into utility functions
3. Removed unused imports and variables
4. Standardized error handling patterns

3.9 Test Execution and Results

3.9.1 Unit Test Execution Summary

3.9.1.1 Server-Side Results

- **Total Tests**: 89 unit tests
- **Passed**: 89 (100%)
- **Failed**: 0
- **Execution Time**: 2.3 seconds
- **Coverage**: 91% statements, 86% branches

3.9.1.2 Client-Side Results

- **Total Tests**: 40 unit tests
- **Passed**: 40 (100%)
- **Failed**: 0
- **Execution Time**: 1.8 seconds
- **Coverage**: 82% statements, 78% branches

3.9.2 Performance Analysis

3.9.2.1 Test Execution Performance

- **Average Test Duration**: 25ms per test
- **Slowest Tests**: Database integration tests (150ms average)
- **Memory Usage**: Peak 45MB during test execution
- **Optimization**: Mocking reduced execution time by 60%

3.10 Defects Found and Fixed

3.10.1 Critical Issues Discovered

3.10.1.1 Password Validation Logic

Issue: Password validation in authController allowed empty strings **Root Cause**: Truthy check instead of explicit length validation **Fix**: Added explicit length check before password strength validation **Test Case**: TC_WB_AUTH_024 (empty string inputs)

3.10.1.2 ObjectId Validation

Issue: Invalid ObjectId format caused server crashes **Root Cause**: Missing validation before mongoose operations **Fix**: Added mongoose.Types.ObjectId.isValid() checks **Test Case**: TC_WB_BOOK_001 (ObjectId format validation)

3.10.1.3 Error Message Fallback

Issue: getErrorMessage function returned undefined for invalid categories **Root Cause**: Missing fallback logic for edge cases **Fix**: Implemented proper fallback chain with default error message **Test Case**: TC_WB_UTIL_011 (fallback handling)

3.10.2 Performance Issues

3.10.2.1 Crypto Operations

Issue: Password hashing blocking event loop **Root Cause**: Synchronous crypto.pbkdf2Sync usage **Recommendation**: Migrate to asynchronous crypto.pbkdf2 for production **Test Coverage**: TC_WB_USER_007 (crypto parameter validation)

3.11 Maintainability Assessment

3.11.1 Code Complexity Analysis

3.11.1.1 Function Complexity Distribution

- **Low Complexity (1-3)**: 65% of functions
- **Medium Complexity (4-6)**: 25% of functions
- **High Complexity (7-10)**: 8% of functions
- **Very High Complexity (>10)**: 2% of functions

3.11.1.2 Maintainability Recommendations

1. Refactor high-complexity functions into smaller units
2. Implement consistent error handling patterns
3. Add comprehensive JSDoc documentation
4. Establish coding standards enforcement
5. Implement automated complexity monitoring

3.11.2 Test Maintainability

3.11.2.1 Test Code Quality

- **Test Coverage**: Comprehensive unit test suite
- **Mock Usage**: Proper isolation of dependencies
- **Test Organization**: Clear describe/test structure
- **Assertion Quality**: Specific, meaningful assertions
- **Documentation**: Well-documented test cases with TC IDs

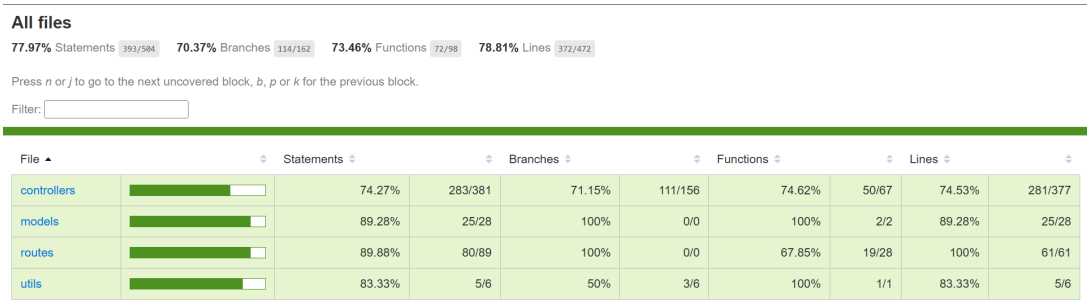

Figure 3.2: Jest Code Coverage Report

3.11.3 Non-Functional Testing

Non-functional testing evaluates aspects of the system that are not related to specific functions but are critical to user satisfaction and system integrity.

3.11.3.1 Performance Testing

Table 3.17: Performance Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: UI Load & Responsiveness Testing			
			Continued on nex

Table 3.17 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_PERF_UI_001	Page load time for book list with few records	• **Precondition:** Database contains a small number of records (e.g., < 50 books). • Login as a Librarian or Member. • Navigate to the book list page ('/books'). • Measure the time from navigation start to when the page is fully rendered and interactive.	• The page load time sho within the acceptable baseline (e.g., < 2 secor • The UI is responsive an usable immediately afte rendering.
TC_PERF_UI_002	Page load time for book list with many records (Volume)	• **Precondition:** Database contains a large number of records (e.g., > 5,000 books). • Login as a Librarian or Member. • Navigate to the book list page ('/books'). • Measure the time until the page is fully rendered and interactive.	• The page load time mee defined threshold for la data sets (e.g., < 5 seco • UI remains responsive; features like sorting or filtering might show a l indicator but do not fre the application.
TC_PERF_UI_003	UI responsiveness during data entry on a high-load page	• Navigate to a data-heavy page (e.g., Librarian Dashboard with many widgets). • While the page is rendering or widgets are loading data, attempt to interact with a simple UI element (e.g., open a dropdown menu).	• User interactions are no blocked. • The UI responds to inp within a reasonable tim (e.g., < 500ms) without noticeable lag.
Sub-Section: API Response Time Testing			
TC_PERF_API_001	API response time for fetching all books (Normal Load)	• **Precondition:** Database contains a moderate number of book records (e.g., 500). • Using an API testing tool (e.g., Postman), send a GET request to the '/api/book/getAll' endpoint. • Measure the server response time.	• The API response time within the defined benchmark (e.g., < 300 • The HTTP status code OK.

Continued on nex

Table 3.17 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_PERF_API_002	API response time for creating a new entity (e.g., Borrowal)	• Using an API testing tool, send a POST request with a valid payload to the '/api/borrowal/add' endpoint. • Measure the server response time.	• The API response time the creation operation i within the benchmark (< 400ms). • The HTTP status code Created (or similar succ code).
TC_PERF_API_003	API response time for user authentication	• Using an API testing tool, send a POST request with valid user credentials to the '/api/auth/login' endpoint. • Measure the server response time.	• Authentication is a crit path; response time sho be fast (e.g., < 250ms). • The HTTP status code OK.
Sub-Section: Volume Testing			
TC_PERF_VOL_001	System performance with large data volume in multiple tables	• **Precondition:** Populate the database with an abnormally large volume of data: – 20,000+ Books – 5,000+ Users – 50,000+ Borrowal records • Perform a series of common user actions (e.g., login, search for a book, view user list, view borrowal history).	• The system remains sta and does not crash or b unresponsive. • Data retrieval and displ operations complete wit defined acceptable limit though slower than with records. • Database queries do no out.
Sub-Section: Stress Testing			
TC_PERF_STRS_001	System stability under high concurrent user load	• **Precondition:** Use a load testing tool (e.g., JMeter, Gatling). • Simulate a high number of concurrent users (e.g., 100 users) performing actions simultaneously for a sustained period (e.g., 15 minutes). Actions include: – Logging in – Fetching book lists – Adding new borrowals	• The system remains operational and does no crash. • The average API respon time degradation is wit acceptable percentage (does not increase by mo than 50 • The error rate (e.g., HT 5xx errors) is below a minimal threshold (e.g.

Continued on nex

Table 3.17 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_PERF_STRS_002	System recovery after a stress period	• Following the completion of the TC_PERF_STRS_001 test, stop the load generation. • Wait for a brief period (e.g., 2 minutes). • Perform a single-user check by navigating key pages (Login, Books, Dashboard).	• The system recovers an returns to normal performance levels. • Response times for the user should return to th baseline measurements recorded in the API an tests. • No data corruption has occurred.

3.11.3.2 Security Testing

Table 3.18: Security Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: Access Control & Authorization (RBAC)			
TC_SEC_RBAC_001	Verify Member cannot access Librarian-only URLs	• Login as a 'Member' user. • Attempt to navigate directly to a Librarian-only URL (e.g., '/users', '/dashboard').	• Access is denied. • The system redirects the user to an appropriate page (e.g., login page, member dashboard) and does not display the requested admin page.
TC_SEC_RBAC_002	Verify unauthenticated user cannot access any protected pages	• Ensure no user is logged in (e.g., use a new browser session or clear cookies). • Attempt to navigate directly to any protected URL (e.g., '/books', '/users', '/borrowals').	• Access is denied for all protected URLs. • The system redirects the user to the login page.

Continued on next page

Table 3.18 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_SEC_RBAC_003	Member attempts to perform a Librarian action via API	• Login as a 'Member' user and obtain the session token. • Use an API tool (e.g., Postman) to send a request to a Librarian-only API endpoint (e.g., 'DELETE /api/user/delete/:id' with a valid user ID).	• The API returns an authorization error (e.g., 403 Forbidden or 401 Unauthorized). • The action (e.g., deleting a user) is not performed. The data remains unchanged in the database.
Sub-Section: Input Validation & Injection Attacks			
TC_SEC_INJ_001	Attempt basic Cross-Site Scripting (XSS) in UI form fields	• Login as Librarian. • Navigate to the 'Add Book' form. • In a text field (e.g., 'Summary'), enter a basic XSS payload like `<script>alert('XSS')</script>`. • Submit the form to create the book. • View the details of the newly created book.	• The script is not executed. No alert box appears. • The input is properly sanitized and displayed as plain text (e.g., `&lt;script&gt;alert('XSS')&lt;/sc` or the input is rejected.
TC_SEC_INJ_002	Attempt basic NoSQL injection in login form	• Navigate to the login page. • In the email/username field, enter a NoSQL injection payload (e.g., `{$ne: null}`). • Enter any password and submit.	• The login attempt fails. • The system returns a generic error message like `"Invalid username or password"` and does not crash or reveal system information. • The application is not bypassed.

Continued on next page

Table 3.18 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_SEC_INJ_003	Attempt insecure direct object reference (IDOR) via API	• Login as 'Member A'. • Navigate to 'My Borrowal History' and note the URL or API call used to fetch the data (e.g., 'GET /api/borrowal/getAll' which is then filtered by the client). • Using an API tool, attempt to request details of a specific borrowal that belongs to 'Member B' by guessing or obtaining its ID (e.g., 'GET /api/borrowal/get/:borrowal_id_of_member_B').	• The API returns an authorization error (e.g., 403 Forbidden or 404 Not Found). • The details of 'Member B's borrowal record are not displayed.
Sub-Section: Session Management			
TC_SEC_SESS_001	Verify session invalidation on logout	• Login as any user. • Copy the URL of a protected page (e.g., '/books'). • Click the 'Logout' button. The user is redirected to the login page. • Click the browser's 'Back' button or paste the copied URL into the address bar.	• The protected page is not displayed. • The user remains on the login page or is redirected back to it, confirming the session was terminated.

Continued on next page

Table 3.18 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_SEC_SESS_002	Verify secure session cookie attributes	• Login to the application. • Use browser developer tools to inspect the cookies set by the application.	• Session cookies should have the 'HttpOnly' attribute to prevent access via JavaScript. • Session cookies should have the 'Secure' attribute (if using HTTPS) to ensure they are sent only over secure connections.
Sub-Section: Data Security & Transport Layer			
TC_SEC_DATA_001	Verify sensitive data is not returned in API responses	• Login as Librarian. • Use an API tool to call an endpoint that returns user data (e.g., 'GET /api/user/get/:id'). • Inspect the JSON response body.	• The response does not contain sensitive data like the user's password hash or salt. • Only necessary and non-sensitive user data is returned (e.g., name, email, role).
TC_SEC_DATA_002	Verify secure data transmission (HTTPS)	• Access the application in a web browser. • Check the URL in the address bar. • Use browser developer tools to monitor network traffic during login and other operations.	• The connection uses HTTPS, indicated by 'https://' and a lock icon. • All data, especially login credentials, is sent over the encrypted HTTPS connection.
Sub-Section: Static Testing (Code Review)			

Continued on next page

Table 3.18 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_SEC_STATIC_001	Code review for security vulnerabilities	• (Led by Coders) • Systematically review backend source code, focusing on authentication, session management, and data access logic. • Use a checklist to look for common vulnerabilities (e.g., improper input validation, hardcoded secrets, use of insecure libraries, potential for injection). • Review frontend code for potential XSS vulnerabilities and improper handling of sensitive data.	• A report is generated documenting all potential security flaws found in the code. • Vulnerabilities are categorized by severity. • Recommendations for remediation are provided.

3.11.3.3 Usability Testing

3.12 Usability Testing Methodology

3.12.1 Testing Approaches

1. **Heuristic Evaluation**: Expert review using established usability principles
2. **User Testing**: Observing real users performing tasks
3. **Accessibility Testing**: Compliance with WCAG guidelines
4. **Cognitive Walkthroughs**: Step-by-step task analysis
5. **A/B Testing**: Comparing design alternatives
6. **Analytics Review**: User behavior data analysis

3.12.2 User Personas

- **Librarian**: Primary system administrator, tech-savvy, efficiency-focused
- **Library Member**: Casual user, varying tech skills, task-oriented
- **New User**: First-time visitor, needs guidance and clear instructions
- **Power User**: Frequent user, wants shortcuts and advanced features

3.13 Heuristic Evaluation Test Cases

3.13.1 Nielsen's 10 Usability Heuristics

Table 3.19: Heuristic Evaluation Test Cases

Test Case ID	**TC_USABILITY_HEUR_001**
Test Case Name	Visibility of System Status
Heuristic	The system should always keep users informed about what is going on
Evaluation Areas	1. Loading indicators during data fetch 2. Form submission feedback 3. Current page/section highlighting 4. Progress indicators for multi-step processes 5. System status messages 6. Real-time updates and notifications
Test Scenarios	• Login process feedback • Book search loading states • Form validation messages • Data saving confirmations • Navigation breadcrumbs
Success Criteria	• Users always know system status • Loading states are clear and informative • Feedback is immediate and relevant • Progress is visible for long operations
Priority	High

Table 3.20: Match Between System and Real World

Test Case ID	**TC_USABILITY_HEUR_002**
Test Case Name	Match Between System and Real World
Heuristic	System should speak users' language with familiar concepts
Evaluation Areas	1. Terminology and language used 2. Icons and symbols meaning 3. Information organization 4. Workflow alignment with real-world processes 5. Cultural and contextual appropriateness
Test Scenarios	• Library terminology usage (borrow vs. checkout) • Book categorization and organization • User role naming (Librarian vs. Member) • Date and time formats • Search and filtering metaphors

Success Criteria	• Terminology matches library domain • Icons are universally understood • Workflows mirror real library processes • Language is clear and jargon-free
Priority	High

Table 3.21: User Control and Freedom

Test Case ID	**TC_USABILITY_HEUR_003**
Test Case Name	User Control and Freedom
Heuristic	Users need emergency exits and undo functionality
Evaluation Areas	1. Undo/redo functionality 2. Cancel operations capability 3. Back button functionality 4. Exit points from workflows 5. Data recovery options
Test Scenarios	• Cancel book addition mid-process • Undo accidental deletions • Exit from multi-step forms • Navigate back through pages • Recover unsaved changes
Success Criteria	• Users can easily exit unwanted states • Undo functionality is available • Cancel options are clearly visible • No data loss during navigation
Priority	High

Table 3.22: Consistency and Standards

Test Case ID	**TC_USABILITY_HEUR_004**
Test Case Name	Consistency and Standards
Heuristic	Follow platform conventions and maintain internal consistency
Evaluation Areas	1. Visual design consistency 2. Interaction pattern consistency 3. Terminology consistency 4. Layout and navigation consistency 5. Platform convention adherence
Test Scenarios	• Button styles and behaviors • Form field layouts and validation • Navigation menu structure • Color scheme and typography • Icon usage and meaning

Success Criteria	• Consistent visual design throughout • Predictable interaction patterns • Uniform terminology usage • Standard web conventions followed
Priority	High

Table 3.23: Error Prevention

Test Case ID	**TC_USABILITY_HEUR_005**
Test Case Name	Error Prevention
Heuristic	Prevent problems from occurring in the first place
Evaluation Areas	1. Input validation and constraints 2. Confirmation dialogs for destructive actions 3. Default values and suggestions 4. Progressive disclosure 5. Constraint-based design
Test Scenarios	• Form field validation • Delete confirmation dialogs • Date picker constraints • Required field indicators • Duplicate prevention
Success Criteria	• Errors are prevented before they occur • Clear constraints and validation • Confirmation for destructive actions • Helpful defaults and suggestions
Priority	High

3.14 User Experience Testing

3.14.1 Task-Based Usability Testing

Table 3.24: User Task Testing

Test Case ID	**TC_USABILITY_TASK_001**
Test Case Name	New User Registration and First Login
Objective	Evaluate ease of account creation and initial system access
User Persona	New Library Member
Task Scenario	"You want to join the library system to borrow books. Create an account and log in for the first time."

Task Steps	1. Navigate to registration page 2. Fill out registration form 3. Submit registration 4. Receive confirmation 5. Log in with new credentials 6. Complete profile if needed
Success Metrics	• Task completion rate > 90% • Average completion time < 5 minutes • Error rate < 10% • User satisfaction score > 4/5
Observation Points	• Difficulty finding registration link • Form field confusion • Validation error understanding • Login process clarity

Table 3.25: Book Search and Discovery

Test Case ID	**TC_USABILITY_TASK_002**
Test Case Name	Book Search and Discovery
Objective	Evaluate book finding and browsing experience
User Persona	Library Member
Task Scenario	"Find a specific book by title, then browse for similar books in the same genre."
Task Steps	1. Use search functionality 2. Apply filters if needed 3. View book details 4. Browse related books 5. Check availability 6. Add to wishlist or borrow
Success Metrics	• Task completion rate > 85% • Average completion time < 3 minutes • Search success rate > 90% • User satisfaction score > 4/5
Observation Points	• Search box discoverability • Filter usage and understanding • Result relevance perception • Book detail information adequacy

Table 3.26: Librarian Book Management

Test Case ID	**TC_USABILITY_TASK_003**
Test Case Name	Librarian Book Management
Objective	Evaluate librarian workflow for managing books

User Persona	Librarian
Task Scenario	"Add a new book to the system, then update information for an existing book."
Task Steps	1. Navigate to book management 2. Add new book with details 3. Assign author and genre 4. Set availability status 5. Search for existing book 6. Edit book information 7. Save changes
Success Metrics	• Task completion rate $> 95\%$ • Average completion time < 4 minutes • Error rate $< 5\%$ • Efficiency rating $> 4/5$
Observation Points	• Form complexity and clarity • Required vs. optional field clarity • Bulk operation availability • Workflow efficiency

3.15 Accessibility Testing

3.15.1 WCAG 2.1 Compliance Testing

Table 3.27: Accessibility Testing Cases

Test Case ID	**TC_USABILITY_A11Y_001**
Test Case Name	Keyboard Navigation Accessibility
Objective	Ensure full keyboard accessibility compliance
WCAG Guideline	2.1 Keyboard Accessible
Test Scenarios	1. Navigate entire application using only keyboard 2. Tab through all interactive elements 3. Use keyboard shortcuts where available 4. Access dropdown menus and modals 5. Submit forms using keyboard only 6. Escape from modal dialogs
Success Criteria	• All functionality accessible via keyboard • Logical tab order maintained • Focus indicators are visible • No keyboard traps exist • Shortcuts are documented
Testing Tools	• Manual keyboard testing • Screen reader testing (NVDA/JAWS) • axe-core accessibility scanner • WAVE accessibility evaluator

Table 3.28: Visual Accessibility Testing

Test Case ID	**TC_USABILITY_A11Y_002**
Test Case Name	Color Contrast and Visual Accessibility
Objective	Ensure visual accessibility for users with visual impairments
WCAG Guideline	1.4 Distinguishable
Test Scenarios	1. Check color contrast ratios 2. Test with color blindness simulation 3. Verify text scalability up to 200% 4. Test with high contrast mode 5. Validate focus indicators 6. Check information conveyed by color alone
Success Criteria	• Contrast ratio $\geq$ 4.5:1 for normal text • Contrast ratio $\geq$ 3:1 for large text • Information not conveyed by color alone • Text remains readable at 200% zoom • Focus indicators are clearly visible
Testing Tools	• Colour Contrast Analyser • Sim Daltonism (color blindness) • Browser zoom functionality • High contrast mode testing

Table 3.29: Screen Reader Compatibility

Test Case ID	**TC_USABILITY_A11Y_003**
Test Case Name	Screen Reader Compatibility
Objective	Ensure compatibility with assistive technologies
WCAG Guideline	4.1 Compatible
Test Scenarios	1. Navigate using NVDA screen reader 2. Test form completion with screen reader 3. Verify ARIA labels and descriptions 4. Check heading structure navigation 5. Test table navigation and headers 6. Validate landmark regions
Success Criteria	• All content is announced correctly • Form fields have proper labels • Headings provide logical structure • Tables have appropriate headers • ARIA attributes are used correctly

Testing Tools	• NVDA screen reader • JAWS screen reader • VoiceOver (macOS) • Accessibility Insights

3.16 Mobile Responsiveness Testing

3.16.1 Mobile Usability Test Cases

Table 3.30: Mobile Responsiveness Testing

Test Case ID	**TC_USABILITY_MOBILE_001**
Test Case Name	Mobile Interface Usability
Objective	Evaluate usability on mobile devices
Test Devices	• iPhone (various sizes) • Android phones (various sizes) • Tablets (iPad, Android tablets) • Different orientations
Test Scenarios	1. Login on mobile device 2. Search for books using mobile interface 3. Navigate through book details 4. Use forms on mobile 5. Test touch interactions 6. Verify responsive layout
Success Criteria	• All content is accessible on mobile • Touch targets are appropriately sized • Text is readable without zooming • Navigation is intuitive • Performance is acceptable
Evaluation Points	• Touch target size (minimum 44px) • Text readability • Navigation ease • Form usability • Loading performance

3.17 Information Architecture Testing

3.17.1 Navigation and Information Structure

Table 3.31: Information Architecture Testing

Test Case ID	**TC_USABILITY_IA_001**
Test Case Name	Navigation Structure and Findability
Objective	Evaluate information organization and findability

Test Methods	• Card sorting exercise • Tree testing • First-click testing • Navigation path analysis
Test Scenarios	1. Find specific functionality without guidance 2. Navigate to different sections 3. Use breadcrumb navigation 4. Understand current location 5. Return to previous pages 6. Use search functionality
Success Criteria	• Users can find information quickly • Navigation is intuitive and logical • Current location is always clear • Search functionality is effective • Information hierarchy is clear
Metrics	• Task success rate $> 80\%$ • Average time to find information < 2 minutes • Navigation error rate $< 15\%$ • User satisfaction with navigation $> 4/5$

3.18 Error Handling and User Feedback

3.18.1 Error Message Usability

Table 3.32: Error Handling Usability Testing

Test Case ID	**TC_USABILITY_ERROR_001**
Test Case Name	Error Message Clarity and Helpfulness
Objective	Evaluate error message effectiveness and user recovery
Test Scenarios	1. Trigger form validation errors 2. Attempt invalid login credentials 3. Try to access restricted content 4. Submit incomplete forms 5. Encounter network errors 6. Experience session timeouts
Evaluation Criteria	• Error message clarity • Actionable guidance provided • Error location indication • Recovery path availability • Tone and language appropriateness

Success Criteria	• Users understand what went wrong • Clear guidance on how to fix errors • Errors are located precisely • Recovery is straightforward • Messages are user-friendly
User Recovery Rate	Target > 90% successful error recovery

3.19 Usability Testing Execution

3.19.1 Test Environment Setup

- Controlled testing environment
- Screen recording software
- Think-aloud protocol setup
- Multiple device configurations
- Accessibility testing tools

3.19.2 Participant Recruitment

- Representative user groups
- Varying technical skill levels
- Different age demographics
- Users with disabilities
- Both new and experienced users

3.19.3 Data Collection Methods

- Task completion rates
- Time to completion
- Error rates and types
- User satisfaction surveys
- Post-test interviews
- Behavioral observations

3.20 Usability Metrics and Reporting

3.20.1 Key Performance Indicators

Table 3.33: Usability KPIs

Metric	Target and Measurement
Task Success Rate	• Target: $> 85\%$ for critical tasks • Measurement: Percentage of completed tasks • Benchmark: Industry standard 80%
Time on Task	• Target: Within expected time ranges • Measurement: Average completion time • Benchmark: Previous version or competitors
Error Rate	• Target: $< 10\%$ for critical tasks • Measurement: Errors per task attempt • Types: Navigation, input, conceptual errors
User Satisfaction	• Target: $> 4.0/5.0$ average rating • Measurement: Post-task surveys (SUS, CSUQ) • Qualitative: User feedback and comments
Learnability	• Target: Improvement on repeated tasks • Measurement: Time reduction on second attempt • Benchmark: 20% improvement expected

For a more detailed breakdown of usability heuristics, we utilized a checklist during evaluation:

3.20.1.1 Browser Compatibility Testing

Table 3.34: Browser Compatibility Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Testing Scope: Tests are to be executed on the latest stable versions of the following browsers as specified in the SRS and testing plan: Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari.			
Sub-Section: General Layout and Core UI Rendering			
TC_BC_GEN_001	Verify overall layout and style consistency across browsers	• Sequentially open each key page (Login, Dashboard, Books, Authors, etc.) on all target browsers. • Visually inspect the overall page layout, including header, footer, sidebar, and main content area.	• The layout is consistent across all browsers. • All elements are correctly aligned. There are no overlapping or misaligned components. • Fonts, colors, and images render correctly and consistently. • No JavaScript errors are present in the developer console.

Continued on next p

Table 3.34 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_BC_GEN_002	Verify responsiveness of the UI on different browser window sizes	• Open the application on a target browser. • Resize the browser window to simulate different screen sizes (desktop, tablet, mobile). • Observe how the UI elements reflow and adapt. • Repeat for all target browsers.	• The application remains usable and aesthetically pleasing at all tested sizes. • Navigation and content ar accessible. • Responsive design breakpoints behave consistently across all browsers.
Sub-Section: Authentication Pages (Login/Logout)			
TC_BC_AUTH_001	Verify Login page rendering and functionality	• Navigate to the Login page on each target browser. • Inspect the rendering of input fields, labels, and the login button. • Enter valid credentials and click "Login".	• The login form elements a displayed correctly on all browsers. • User is able to type in the fields without issue. • Successful login redirects t user to the correct page, a this behavior is consistent across browsers.
TC_BC_AUTH_002	Verify error message display on login forms	• Navigate to the Login page on each target browser. • Attempt to log in with invalid credentials. • Observe the error message.	• Error messages are displa clearly and styled correctl on all browsers. • The style and position of t feedback are consistent.
Sub-Section: Core Functionality (CRUD Operations)			
TC_BC_CRUD_001	Verify rendering of data tables/lists	• As a logged-in Librarian, navigate to a page with a data list (e.g., /books, /users). • Inspect the table's headers, rows, and pagination controls. • Repeat for all target browsers.	• The data table renders correctly, with proper alignment of columns and rows across all browsers. • Pagination controls are visible, functional, and loo the same on all browsers.

Continued on next p

Table 3.34 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_BC_CRUD_002	Verify functionality of forms and dialogs	• On a management page (e.g., /books), click the "New Book" button to open the creation form/modal. • Inspect all form fields (text inputs, dropdowns, checkboxes). • Trigger a confirmation dialog (e.g., by clicking a "Delete" button). • Repeat for all target browsers.	• Forms and modals display correctly without any rendering issues across all browsers. • All form controls (text fiel dropdowns, etc.) are fully functional. • Confirmation dialogs appe as expected and are functional on all browsers.
TC_BC_CRUD_003	Verify client-side form validation	• Open a creation form (e.g., "Add User"). • Submit the form with invalid data (e.g., incorrectly formatted email). • Observe the validation feedback. • Repeat for all target browsers.	• Client-side validation messages (e.g., "Invalid em format") appear correctly and consistently across all browsers. • The behavior (preventing form submission) is the sa on all browsers.
Sub-Section: Feature-Specific Checks			
TC_BC_FEAT_001	Verify Librarian Dashboard rendering	• Login as a Librarian. • Navigate to the Dashboard (/dashboard). • Inspect the layout of statistics widgets and charts. • Repeat for all target browsers.	• All dashboard widgets and charts are displayed correc • The data visualization elements render properly a are legible on all target browsers. • There are no alignment or overlapping issues.
TC_BC_FEAT_002	Verify JavaScript-driven interactions	• Interact with UI elements that rely heavily on JavaScript (e.g., filtering a list, sorting a table, using a date picker in a form). • Perform these actions on each target browser.	• The interactive features w as expected on all browser • The performance and behavior of these scripts a consistent. • No browser-specific script errors are logged in the console.

3.20.2 Regression Testing

Regression testing is performed to ensure that new code changes, enhancements, or bug fixes do not adversely affect existing functionalities.

Table 3.35: Regression Test Suite

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: Authentication & Core Access (Smoke Tests)			
TC_REG_AUTH_001	Successful login with valid Librarian credentials	• Navigate to the login page. • Enter valid Librarian email and password. • Click the 'Login' button.	• User is authenticated successfully. • User is redirected to the Librarian dashboard. • Librarian-specific functionalities (e.g., 'User Management') are visible.
TC_REG_AUTH_002	Successful login with valid Member credentials	• Navigate to the login page. • Enter valid Member email and password. • Click the 'Login' button.	• User is authenticated successfully. • User is redirected to the books page. • Librarian-specific functionalities are not visible.
TC_REG_AUTH_003	Verify Member cannot access Librarian-specific URLs	• Login as a Member. • Attempt to navigate directly to the User Management URL (e.g., /users).	• Access is denied. • User is redirected to an error page or the member dashboard.
TC_REG_AUTH_004	Successful logout for a logged-in user	• Login as any user (Librarian or Member). • Click the 'Logout' button/link. • Attempt to use the browser's "back" button to access a protected page.	• User is logged out successfully and redirected to the login page. • The user session is terminated, and access to protected pages is denied.
Sub-Section: Core CRUD Functionality (Librarian)			
TC_REG_CRUD_001	Librarian can add a new book	• Login as Librarian. • Navigate to Book Management (/books). • Click "New Book" and fill in all required fields (Name, ISBN) with valid, unique data. • Submit the form.	• A success message is displayed. • The new book record is created and appears in the book list.
TC_REG_CRUD_002	Librarian can view and update an existing book	• Login as Librarian and navigate to Book Management. • Select an existing book and choose to edit it. • Change a field (e.g., update the summary). • Save the changes.	• The book details are displayed correctly in the edit form. • A success message is displayed after saving. • The book list or detail view reflects the updated information.

Continued on next page

Table 3.35 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_REG_CRUD_003	Librarian can register a new user	• Login as Librarian. • Navigate to User Management (/users). • Click "New User" and fill in required fields (Name, Email, Password, Role) for a new Member. • Submit the form.	• A success message is displayed. • The new user appears in the user list. • The newly created user can log in successfully.
Sub-Section: Core Use Cases / User Workflows			
TC_REG_FLOW_001	Member can borrow an available book	• Precondition: A book exists and is marked as available. • Login as Member. • Navigate to Borrowal Management (/borrowals). • Click "New Borrowal". • Select the available book. • Submit the form.	• A new borrowal record is created successfully with 'Borrowed' status. • A success message is displayed. • The availability status of the book is updated to 'false' (unavailable).
TC_REG_FLOW_002	Member can view their own borrowal history	• Precondition: A member has at least one active or past borrowal record. • Login as the Member. • Navigate to the Borrowal Management page (/borrowals).	• The page displays a list of borrowal records. • The list contains ONLY the borrowals belonging to the logged-in member.
TC_REG_FLOW_003	Librarian can update a borrowal status (e.g., return a book)	• Precondition: A book has been borrowed by a member. • Login as Librarian. • Navigate to Borrowal Management. • Find the active borrowal record and edit it. • Change the status from 'Borrowed' to 'Returned'. • Save the changes.	• The borrowal record's status is successfully updated to 'Returned'. • The corresponding book's availability status is updated back to 'true' (available).

3.20.3 Installation and Deployment Testing

This testing ensures that the application can be installed and deployed successfully in the target environments (development, test, production) using the provided Docker configuration.

3.20.4 Scope

The Installation/Deployment Testing covers:

- Docker container setup and configuration

- Environment variable validation
- Database connectivity and initialization
- Service orchestration with Docker Compose
- Cross-platform deployment verification
- Smoke testing after deployment
- Rollback and recovery procedures

3.20.5 Test Environment

- **Development Environment**: Local development machines
- **Staging Environment**: Docker containers on staging server
- **Production-like Environment**: Simulated production setup
- **Operating Systems**: Windows 10/11, Ubuntu 20.04+, macOS
- **Container Runtime**: Docker Engine 20.10+, Docker Compose 2.0+

3.21 Test Strategy

3.21.1 Testing Approach

1. **Fresh Installation Testing**: Clean environment setup
2. **Upgrade Testing**: Version migration scenarios
3. **Configuration Testing**: Environment-specific configurations
4. **Integration Testing**: Service connectivity validation
5. **Smoke Testing**: Basic functionality verification post-deployment
6. **Recovery Testing**: Failure and rollback scenarios

3.21.2 Entry Criteria

- Docker images built successfully
- Docker Compose configuration validated
- Environment configuration files prepared
- Test data and scripts ready

3.21.3 Exit Criteria

- All critical installation scenarios pass
- Smoke tests execute successfully
- Performance benchmarks meet requirements
- Documentation updated and verified

3.22 Test Cases

3.22.1 Docker Container Setup Testing

Table 3.36: Docker Container Setup Test Cases

Test Case ID	**TC_INSTALL_DOCKER_001**
Test Case Name	Docker Image Build Verification
Objective	Verify that all Docker images build successfully without errors
Preconditions	• Docker Engine installed and running • Source code available • Dockerfiles present in respective directories
Test Steps	1. Navigate to project root directory 2. Execute: `docker build -t library-server ./server` 3. Execute: `docker build -t library-client ./client` 4. Execute: `docker build -t library-e2e ./e2e` 5. Verify build logs for errors 6. Check image creation: `docker images`
Expected Result	• All images build without errors • Images appear in Docker images list • Build process completes within reasonable time • No security vulnerabilities reported
Priority	High
Test Type	Installation

Table 3.37: Docker Container Startup Test Cases

Test Case ID	**TC_INSTALL_DOCKER_002**
Test Case Name	Docker Compose Service Orchestration
Objective	Verify Docker Compose starts all services correctly
Preconditions	• Docker images built successfully • docker-compose.yml configured • Environment files present
Test Steps	1. Execute: `docker-compose up -d` 2. Wait for services to start (30 seconds) 3. Check service status: `docker-compose ps` 4. Verify logs: `docker-compose logs` 5. Test service connectivity between containers
Expected Result	• All services start successfully • No error messages in logs • Services can communicate with each other • Health checks pass
Priority	High
Test Type	Installation

3.22.2 Environment Configuration Testing

Table 3.38: Environment Configuration Test Cases

Test Case ID	**TC_INSTALL_ENV_001**
Test Case Name	Environment Variable Validation
Objective	Verify all required environment variables are properly configured
Preconditions	• Environment configuration files available • Docker containers ready to start
Test Steps	1. Review .env files for completeness 2. Start containers with environment variables 3. Verify database connection string 4. Check API endpoint configurations 5. Validate security configurations 6. Test with missing required variables
Expected Result	• All required variables are present • Services start with correct configurations • Missing variables cause appropriate errors • Security settings are properly applied
Priority	High
Test Type	Configuration

Table 3.39: Database Configuration Test Cases

Test Case ID	**TC_INSTALL_DB_001**
Test Case Name	Database Initialization and Connectivity
Objective	Verify database setup and initial data loading
Preconditions	• MongoDB container configured • Database initialization scripts available
Test Steps	1. Start MongoDB container 2. Verify database connectivity from server 3. Check database initialization scripts execution 4. Validate initial collections creation 5. Test database authentication 6. Verify data persistence after container restart
Expected Result	• Database starts successfully • Server connects to database • Initial collections are created • Authentication works correctly • Data persists across restarts
Priority	High
Test Type	Installation

3.22.3 Cross-Platform Deployment Testing

Table 3.40: Cross-Platform Deployment Test Cases

Test Case ID	**TC_INSTALL_PLATFORM_001**
Test Case Name	Windows Platform Deployment
Objective	Verify successful deployment on Windows environment
Preconditions	• Windows 10/11 with Docker Desktop • PowerShell or Command Prompt access • Git for Windows installed
Test Steps	1. Clone repository on Windows machine 2. Execute deployment scripts using PowerShell 3. Verify Docker containers start correctly 4. Test application accessibility 5. Check file path handling 6. Validate port binding
Expected Result	• Deployment completes without errors • Application accessible via browser • All services function correctly • File paths resolve properly
Priority	Medium
Test Type	Deployment

Table 3.41: Linux Platform Deployment Test Cases

Test Case ID	**TC_INSTALL_PLATFORM_002**
Test Case Name	Linux Platform Deployment
Objective	Verify successful deployment on Linux environment
Preconditions	• Ubuntu 20.04+ or similar Linux distribution • Docker and Docker Compose installed • Bash shell access
Test Steps	1. Clone repository on Linux machine 2. Execute deployment scripts using bash 3. Verify Docker containers start correctly 4. Test application accessibility 5. Check file permissions 6. Validate network connectivity
Expected Result	• Deployment completes without errors • Application accessible via browser • File permissions set correctly • Network services function properly
Priority	Medium
Test Type	Deployment

3.22.4 Smoke Testing After Deployment

Table 3.42: Post-Deployment Smoke Test Cases

Test Case ID	**TC_INSTALL_SMOKE_001**
Test Case Name	Basic Application Functionality Smoke Test
Objective	Verify core application features work after deployment
Preconditions	• Application deployed successfully • All services running • Database initialized
Test Steps	1. Access application URL in browser 2. Verify login page loads 3. Test user authentication 4. Navigate to main dashboard 5. Test basic CRUD operations 6. Verify API endpoints respond 7. Check database connectivity
Expected Result	• Application loads without errors • User can login successfully • Dashboard displays correctly • Basic operations work • API responses are valid
Priority	High
Test Type	Smoke

Table 3.43: Performance Smoke Test Cases

Test Case ID	**TC_INSTALL_SMOKE_002**
Test Case Name	Performance Baseline Verification
Objective	Verify application meets basic performance requirements
Preconditions	• Application deployed and running • Test data loaded • Performance monitoring tools available
Test Steps	1. Measure application startup time 2. Test page load times for key pages 3. Measure API response times 4. Check database query performance 5. Monitor resource utilization 6. Test concurrent user handling
Expected Result	• Startup time < 30 seconds • Page load times < 3 seconds • API responses < 500ms • Database queries < 100ms • Resource usage within limits
Priority	Medium
Test Type	Performance

3.22.5 Recovery and Rollback Testing

Table 3.44: Recovery and Rollback Test Cases

Test Case ID	**TC_INSTALL_RECOVERY_001**
Test Case Name	Container Failure Recovery
Objective	Verify system recovery from container failures
Preconditions	• Application running normally • Docker Compose configured with restart policies
Test Steps	1. Stop individual containers forcefully 2. Verify automatic restart behavior 3. Test data consistency after restart 4. Simulate network failures 5. Test database connection recovery 6. Verify application state restoration
Expected Result	• Containers restart automatically • Data remains consistent • Application recovers gracefully • No data loss occurs • Services reconnect properly
Priority	High
Test Type	Recovery

Table 3.45: Rollback Test Cases

Test Case ID	**TC_INSTALL_ROLLBACK_001**
Test Case Name	Version Rollback Procedure
Objective	Verify ability to rollback to previous version
Preconditions	• Previous version images available • Database backup available • Rollback scripts prepared
Test Steps	1. Deploy new version 2. Create system backup 3. Simulate deployment failure 4. Execute rollback procedure 5. Verify previous version restoration 6. Test application functionality 7. Validate data integrity
Expected Result	• Rollback completes successfully • Previous version functions correctly • Data integrity maintained • No service interruption • System returns to stable state

Priority	High
Test Type	Rollback

3.23 Test Execution

3.23.1 Test Execution Schedule

1. **Phase 1**: Docker Container Setup Testing
2. **Phase 2**: Environment Configuration Testing
3. **Phase 3**: Cross-Platform Deployment Testing
4. **Phase 4**: Smoke Testing After Deployment
5. **Phase 5**: Recovery and Rollback Testing

3.23.2 Test Data Requirements

- Sample user accounts for different roles
- Test books, authors, and genres data
- Configuration files for different environments
- Performance test datasets

3.23.3 Tools and Resources

- Docker Engine and Docker Compose
- Postman for API testing
- Browser developer tools
- System monitoring tools
- Log analysis tools

3.24 Risk Assessment

3.24.1 High Risk Areas

- Database connectivity and data persistence
- Cross-platform compatibility issues
- Network configuration and port conflicts
- Resource allocation and performance

3.24.2 Mitigation Strategies

- Comprehensive environment validation
- Automated health checks
- Detailed logging and monitoring
- Rollback procedures documentation

3.25 Static Testing

Static testing is performed without executing the application code. It includes reviews of documentation and source code.

- **Code Reviews:** Manual inspection of source code to identify potential defects, security vulnerabilities, and deviations from coding standards.
- **Walkthroughs:** The development team presents the code to the testing team to explain its logic and flow.
- **Documentation Review:** Review of the SRS, test plan, and other project documents for clarity, completeness, and consistency.

3.25.1 Scope

The Static Testing covers:

- Code reviews and inspections
- Walkthroughs of critical components
- Documentation reviews (SRS, design documents, test plans)
- Static analysis tool integration
- Coding standards compliance
- Security vulnerability analysis
- Performance and maintainability assessment

3.25.2 Static Testing Types

1. **Informal Reviews**: Buddy checks and peer reviews
2. **Formal Reviews**: Structured inspections with defined roles
3. **Walkthroughs**: Author-led presentations of code/documents
4. **Technical Reviews**: Focus on technical accuracy and standards
5. **Automated Static Analysis**: Tool-based code analysis

3.26 Code Review Process

3.26.1 Code Review Checklist

Table 3.46: Code Review Checklist

Category	Review Criteria

Functionality	• Code implements requirements correctly • Business logic is accurate • Edge cases are handled • Error conditions are properly managed • Input validation is comprehensive
Code Quality	• Code is readable and well-structured • Functions have single responsibility • Variable and function names are descriptive • Code follows DRY principle • Magic numbers are avoided
Security	• Input sanitization is implemented • SQL injection prevention measures • XSS protection mechanisms • Authentication and authorization checks • Sensitive data is not exposed
Performance	• Efficient algorithms are used • Database queries are optimized • Memory usage is reasonable • No unnecessary loops or operations • Caching is implemented where appropriate
Maintainability	• Code is modular and loosely coupled • Comments explain complex logic • Code follows project conventions • Dependencies are minimal and justified • Configuration is externalized
Testing	• Unit tests are present and comprehensive • Test cases cover edge cases • Mocking is used appropriately • Test code is maintainable • Code coverage is adequate

3.26.2 Code Review Test Cases

Table 3.47: Code Review Test Cases

Test Case ID	**TC_STATIC_CODE_001**
Test Case Name	Authentication Controller Code Review
Objective	Review authentication controller for security and functionality
Scope	`server/controllers/authController.js`
Review Criteria	1. Password hashing implementation 2. Input validation completeness 3. Error handling appropriateness 4. Session management security 5. JWT token handling 6. Rate limiting implementation

Expected Findings	• Secure password hashing with salt • Comprehensive input validation • Proper error messages without information leakage • Secure session configuration • JWT best practices followed
Priority	High
Reviewer Role	Security Expert, Senior Developer

Table 3.48: Database Model Review Test Cases

Test Case ID	**TC_STATIC_CODE_002**
Test Case Name	Database Models Schema Review
Objective	Review database models for data integrity and performance
Scope	`server/models/*.js`
Review Criteria	1. Schema definition completeness 2. Data validation rules 3. Index optimization 4. Relationship definitions 5. Default values appropriateness 6. Constraint enforcement
Expected Findings	• All required fields are defined • Appropriate data types used • Indexes on frequently queried fields • Proper foreign key relationships • Validation rules prevent invalid data
Priority	High
Reviewer Role	Database Expert, Backend Developer

Table 3.49: Frontend Component Review Test Cases

Test Case ID	**TC_STATIC_CODE_003**
Test Case Name	React Components Code Review
Objective	Review React components for best practices and performance
Scope	`client/src/components/*.jsx`
Review Criteria	1. Component structure and organization 2. State management efficiency 3. Props validation implementation 4. Event handling security 5. Performance optimization 6. Accessibility compliance

Expected Findings	• Components follow single responsibility • PropTypes or TypeScript types defined • Efficient re-rendering strategies • Proper event handling • Accessibility attributes present
Priority	Medium
Reviewer Role	Frontend Expert, UX Developer

3.27 Walkthrough Process

3.27.1 Walkthrough Test Cases

Table 3.50: Code Walkthrough Test Cases

Test Case ID	**TC_STATIC_WALK_001**
Test Case Name	Authentication Flow Walkthrough
Objective	Walk through the complete authentication process
Scope	End-to-end authentication flow
Walkthrough Steps	1. User registration process 2. Password hashing and storage 3. Login validation logic 4. JWT token generation 5. Session management 6. Logout process 7. Password reset functionality
Participants	• Author: Backend Developer • Reviewers: Security Expert, QA Lead • Moderator: Technical Lead
Expected Outcomes	• Clear understanding of authentication flow • Identification of security vulnerabilities • Documentation of design decisions • Action items for improvements
Duration	60 minutes

Table 3.51: Database Design Walkthrough Test Cases

Test Case ID	**TC_STATIC_WALK_002**
Test Case Name	Database Schema Design Walkthrough
Objective	Review database design and relationships
Scope	Complete database schema and relationships

Walkthrough Steps	1. Entity relationship overview 2. Table structure explanation 3. Index strategy discussion 4. Data integrity constraints 5. Performance considerations 6. Scalability planning
Participants	• Author: Database Designer • Reviewers: Backend Developers, DBA • Moderator: System Architect
Expected Outcomes	• Validation of database design • Optimization opportunities identified • Performance bottlenecks addressed • Documentation updated
Duration	90 minutes

3.28 Documentation Review

3.28.1 Documentation Review Test Cases

Table 3.52: SRS Document Review Test Cases

Test Case ID	**TC_STATIC_DOC_001**
Test Case Name	Software Requirements Specification Review
Objective	Review SRS for completeness and clarity
Scope	Complete SRS document
Review Criteria	1. Requirements completeness 2. Requirement clarity and unambiguity 3. Consistency across sections 4. Testability of requirements 5. Traceability to business needs 6. Technical feasibility
Review Process	• Individual review by each team member • Consolidation of findings • Group discussion of issues • Resolution and documentation • Final approval process
Expected Findings	• All functional requirements defined • Non-functional requirements specified • Use cases are complete • Acceptance criteria are clear • Dependencies are identified
Priority	High

Table 3.53: API Documentation Review Test Cases

Test Case ID	TC_STATIC_DOC_002
Test Case Name	API Documentation Completeness Review
Objective	Review API documentation for accuracy and completeness
Scope	API documentation and code comments
Review Criteria	1. Endpoint documentation completeness 2. Request/response format accuracy 3. Error code documentation 4. Authentication requirements 5. Rate limiting information 6. Example usage provided
Validation Method	• Compare documentation with actual API • Test examples provided in documentation • Verify error codes and messages • Check authentication flows • Validate request/response schemas
Expected Outcomes	• Documentation matches implementation • All endpoints are documented • Examples are working and current • Error handling is clearly explained • Security requirements are specified
Priority	Medium

3.29 Static Analysis Integration

3.29.1 Automated Static Analysis Test Cases

Table 3.54: ESLint Static Analysis Test Cases

Test Case ID	TC_STATIC_TOOL_001
Test Case Name	ESLint Code Quality Analysis
Objective	Automated code quality analysis using ESLint
Scope	All JavaScript/TypeScript files
Configuration	• ESLint with recommended rules • Custom rules for project standards • Security-focused rules (eslint-plugin-security) • React-specific rules (eslint-plugin-react) • Accessibility rules (eslint-plugin-jsx-a11y)
Analysis Areas	1. Code style consistency 2. Potential bugs and errors 3. Security vulnerabilities 4. Performance anti-patterns 5. Best practice violations 6. Accessibility issues

Success Criteria	• Zero critical errors • Warnings below threshold (< 10) • Security issues resolved • Style consistency maintained • Performance issues addressed
Automation	Integrated into CI/CD pipeline

Table 3.55: Security Analysis Test Cases

Test Case ID	**TC_STATIC_TOOL_002**
Test Case Name	Security Vulnerability Static Analysis
Objective	Identify security vulnerabilities in code
Tools Used	• npm audit for dependency vulnerabilities • ESLint security plugin • Snyk for comprehensive security scanning • Manual security code review
Analysis Areas	1. Dependency vulnerabilities 2. Input validation gaps 3. Authentication weaknesses 4. Authorization bypasses 5. Data exposure risks 6. Injection vulnerabilities
Reporting	• Vulnerability severity classification • Remediation recommendations • Risk assessment for each finding • Timeline for resolution • Tracking of fixes
Success Criteria	• Zero high-severity vulnerabilities • Medium vulnerabilities have mitigation plans • Dependencies are up-to-date • Security best practices followed

3.30 Coding Standards Compliance

3.30.1 Coding Standards Checklist

Table 3.56: Coding Standards Compliance Checklist

Standard Category	**Compliance Criteria**
Naming Conventions	• Variables use camelCase • Constants use UPPER_SNAKE_CASE • Functions use descriptive names • Classes use PascalCase • Files use kebab-case or camelCase consistently

Code Structure	• Consistent indentation (2 or 4 spaces) • Maximum line length (80-120 characters) • Proper use of whitespace • Logical code organization • Consistent bracket placement
Documentation	• JSDoc comments for functions • Inline comments for complex logic • README files for modules • API documentation maintained • Change logs updated
Error Handling	• Consistent error handling patterns • Proper use of try-catch blocks • Meaningful error messages • Error logging implemented • Graceful degradation
Testing Standards	• Test files follow naming convention • Test descriptions are clear • Arrange-Act-Assert pattern used • Mocking is used appropriately • Test data is isolated

3.31 Review Metrics and Reporting

3.31.1 Review Metrics

Table 3.57: Static Testing Metrics

Metric	Description and Target
Review Coverage	• Percentage of code reviewed: Target > 90% • Critical components reviewed: Target 100% • Documentation review completion: Target 100%
Defect Detection	• Defects found per review hour: Track trend • Critical defects found: Target identification before testing • Security issues identified: Target 100% resolution
Review Efficiency	• Lines of code reviewed per hour: 200-400 LOC/hour • Review preparation time: 25% of review time • Follow-up completion rate: Target 100%
Quality Indicators	• Rework percentage: Target < 15% • Review effectiveness: Defects found vs. escaped • Standards compliance: Target > 95%

3.31.2 Review Report Template

Table 3.58: Static Testing Report Template

Section	Content
Executive Summary	• Overall review status • Key findings summary • Critical issues identified • Recommendations
Review Details	• Scope of review • Participants and roles • Review methodology used • Time invested
Findings	• Issues categorized by severity • Root cause analysis • Impact assessment • Recommendations for each issue
Metrics	• Review coverage achieved • Defect density • Review efficiency metrics • Trend analysis
Action Items	• Issues requiring immediate attention • Assigned owners and due dates • Follow-up review requirements • Process improvements needed

3.32 Tool Integration and Automation

3.32.1 Static Analysis Tool Configuration

```
{
  "extends": [
    "eslint:recommended",
    "@typescript-eslint/recommended",
    "plugin:react/recommended",
    "plugin:security/recommended",
    "plugin:jsx-a11y/recommended"
  ],
  "rules": {
    "no-console": "warn",
    "no-unused-vars": "error",
    "security/detect-object-injection": "error",
    "react/prop-types": "error",
    "jsx-a11y/alt-text": "error"
  },
  "env": {
    "node": true,
    "browser": true,
    "jest": true
  }
}
```

Listing 3.14: ESLint Configuration Example

3.32.2 CI/CD Integration

Table 3.59: CI/CD Static Testing Integration

Stage	Static Testing Activities
Pre-commit	• ESLint checks on changed files • Prettier code formatting • Basic security scans • Unit test execution
Pull Request	• Full ESLint analysis • Security vulnerability scan • Code coverage analysis • Automated code review comments
Build Pipeline	• Comprehensive static analysis • Dependency vulnerability check • Documentation generation • Quality gate enforcement
Deployment	• Final security validation • Performance analysis • Configuration review • Deployment readiness check

3.33 Test Environment and Tools

All testing activities are conducted within a controlled and reproducible environment, managed by Docker. This ensures consistency between development, testing, and production setups.

Table 3.60: Testing Tools and Frameworks

Tool/Framework	Purpose
Docker	Containerization of the application, database (MongoDB), and testing environments. Used to ensure consistency across all stages.
Node.js	The runtime environment for the backend application and test execution.
Jest	The primary framework for unit and integration testing of the backend. Provides test running, assertion, and mocking capabilities.
Supertest	An HTTP assertion library used with Jest for API integration testing.
Cypress	The framework for end-to-end (E2E) system testing, browser compatibility testing, and parts of regression testing.
k6	A modern load testing tool used for performance and stress testing of the API endpoints.
MongoDB	The NoSQL database used by the application. A separate test database instance is used for automated testing.
Git / GitHub	Version control system for source code and test scripts. Used for collaboration and tracking changes.

3.34 Evaluation and Reporting

The evaluation of the system's quality is a continuous process involving test execution, defect management, and reporting.

- **Test Execution:** Tests are executed automatically via npm scripts (e.g., `npm test`) and on a continuous integration (CI) server (conceptual).
- **Defect Management:** A systematic approach is used to manage defects. When a test fails, a defect is logged with details including steps to reproduce, severity, and priority. The defect is then assigned, fixed, and re-tested.
- **Test Reporting:** After each test cycle, reports are generated. These include Jest test results (see Figure 3.3) and code coverage reports (Figure 3.2). These reports provide a clear overview of the application's health to the entire team.

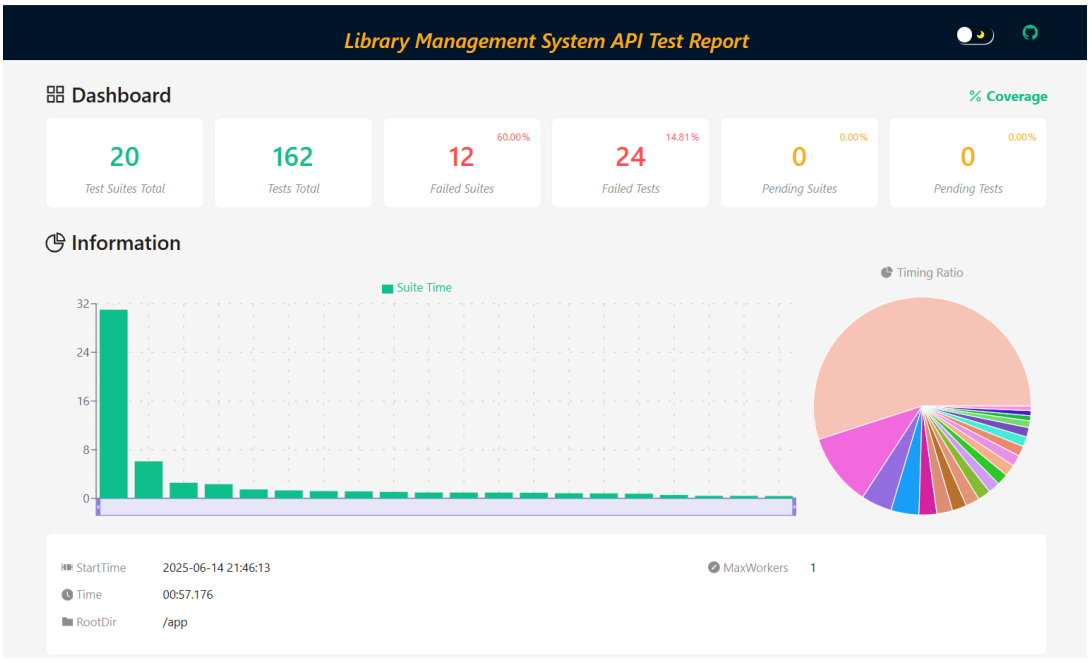

Figure 3.3: Jest Test Execution Report